

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_0febe894-ff9\agent-  
a6c3ecd76fb3f57.jsonl`  
- SHA-256: `9c0fbcabfd023de6f2bee1219b0add442a12a572e7c25fa8f9947a70b2f43f46`  
- Source modified: `2026-07-07T03:41:25+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_0febe894-ff9`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-07T03:35:16.488Z)

Reviewing GitHub PR #58 "docs: refresh current project documentation" (branch agent/docs-sweep
-> main). DOCS-ONLY refresh + a few
doc-accuracy source edits (help text, docstrings, Dockerfile comment, pyproject description).
+782/-787, 14 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr58.
This is an ACCURACY review: the docs must match the MERGED code (Tracks B-F + ADR 0012 F1-F7 are
all merged). Read the doc diff:

```
git -C C:/Users/avidu/Projects/Telegram diff 7fadffb..origin/agent/docs-sweep -- <file>  
and cross-check every concrete claim against the actual code (src/tg_video_filter/*.py,  
docs/adr/*). Read code via Read/Grep; run  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check defaults/behavior.
```

KEY GROUND TRUTH (verified by the main reviewer) to check the docs against:

- Selective scanning is OPT-IN: new sources default watched=false (should_ingest_source); the
watched flag is user-controlled (set_source_watched / /watch,/unwatch);
upsert_source no longer clobbers watched (a fix in #41). Commands: /channels, /watch
<id|@name>, /unwatch, /watchlist.
- FilterConfig exclusion DEFAULTS: exclude_gif=True, exclude_round=True, exclude_nosound=False
(i.e. GIF/round excluded, nosound allowed). The old "all off by default" was WRONG.
- Policy pipeline stages (evaluate_pipeline order): topic-preferences -> url-reputation ->
caption-safety -> deep-inspection. Deep inspection is a budget BOUNDARY only; NO production
fetcher is wired (ctx.fetcher/budget are None in the live path), so enabling deep_inspect drops
items.
- Config lives in policies.config_json (raw dict); save_filter_config MERGES flat FilterConfig
keys so ADR 0012 keys (topics/security_checks/deep_inspect) survive a /set. There is NO /set
command to set those policy keys yet (dormant).
- Tables: accounts, telegram_sessions, sources, content_items, policies, destinations,
routing_rules, delivery_attempts, policy_decisions (+ *_deprecated). policy_decisions has NO
TTL/retention cleanup (deferred, issue #56).
- Dedup: 3-tier (message instance, Telegram document.id, metadata fingerprint) via
content_items.dedup_key. Delivery modes forward/link/both to me/@username/chat id.
- /why <content_item_id> is account-scoped (get_content_item_for_account), renders the stored
policy trace or a caveated metadata replay; markdown-escaped.

Ground rules: READ the doc diff AND the code, cite file:line. Report ONLY doc claims in THIS PR
diff that are FACTUALLY WRONG or MISLEADING vs the merged code
(a stale table name, a command that does not exist, a wrong default, a wrong ADR status, a "does
X" the code does not do). No style nitpicks unless misleading.
Severity: high (a wrong claim a user/operator would act on and be broken by, e.g. wrong
command/flag/deploy step), medium (misleading/ambiguous/stale), low (typo/broken link/minor).

DIMENSION: verify docs/plan.md and docs/runbook.md against the code. Check: build-status claims
(is anything marked done that is not, or vice versa?), the DEPLOY/operational steps (do the
commands, env vars, paths, docker steps, bootstrap flow actually match config.py / Dockerfile /
docker-compose / __main__ / bootstrap.py?), the command surface a user is told to run (do

/watch, /why, /backfill, /set exist as described?), and any stale table/phase references. An operator following a wrong runbook step is high severity.

2. assistant (2026-07-07T03:35:18.972Z)

I'll review PR #58 for documentation accuracy against the merged code. Let me start by reading the doc diff and cross-checking against the actual code.

3. user (2026-07-07T03:35:21.555Z)

```
3069f18 docs: refresh current project documentation
7fadffb feat(commands): add why explainability command (#57)
6149c38 feat(policy): persist policy decision traces (#55)
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
---FILES---
Dockerfile
README.md
docs/adr/0007-inline-config-commands.md
docs/adr/0009-selective-channel-scanning.md
docs/adr/0010-identity-and-session-separation.md
docs/adr/0011-provider-neutral-domain-schema.md
docs/adr/0012-policy-pipeline.md
docs/design.md
docs/plan.md
docs/runbook.md
pyproject.toml
src/tg_video_filter/command_service.py
src/tg_video_filter/models.py
tests/test_commands.py
```

4. assistant (2026-07-07T03:35:24.100Z)

Let me read the doc diffs and gather the ground-truth code in parallel.

5. user (2026-07-07T03:35:26.274Z)

<persisted-output>
Output too large (45.5KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tool-results\bo4y02nr.txt

```
Preview (first 2KB):
===== DIFF: docs/plan.md =====
diff --git a/docs/plan.md b/docs/plan.md
index 07b330c..9c0c1f1 100644
--- a/docs/plan.md
+++ b/docs/plan.md
@@ -1,227 +1,113 @@
-# Plan: Telegram Video Filter Userbot (v1)
-
-## TL;DR
-A Telethon-based Telegram **userbot** that logs in as the user's own account, watches
-new video messages across all channels/groups the account is a member of, applies
-user-configured filters (duration, resolution, file size), and **forwards matching
-videos to the user's Saved Messages** in real time. Python + Telethon + SQLite +
-Docker, multi-user capable (<100), with a docs folder.
-
-v1 = lean. v2 = digest delivery, on-demand `/search`, public-join + manual-add
-ingestion paths, more filters (codec/keyword/mime).
-
-## Decisions (locked)
-- **Stack**: Python 3.11+ · Telethon (MTProto userbot) · SQLite (aiosqlite) · Docker
-- **Ingestion (v1)**: Userbot only ? Telethon reads all of the user's channel subscriptions via
```

```

`events.NewMessage`. No Bot API, no public-join, no manual-add in v1.
-- **Delivery (v1)**: Real-time forward of each qualifying video to the user's Saved Messages
(`client.forward_messages` to `me`).
-- **Filters (v1)**: duration (min/max seconds), resolution (height==1080 etc.), file size
(min/max MB). Engine designed to be extensible.
-- **Probing**:: Trust Telegram message metadata first (`message.video` +
`DocumentAttributeVideo`); `/inspect` ffprobe-on-demand is a v2 nice-to-have, not in v1 scope.
-- **Scale**:: Multi-user, small (<100). Per-user Telethon session, per-user filter config, per-
user seen-state.
-- **Tooling**:: git, `.gitignore`, README, Dockerfile + docker-compose, pytest scaffold, docs
folder (design/ADRs/runbook), pre-commit (ruff).
-- **Phased**:: v1 lean now; v2 defers extra ingestion paths, digest, search, more filters.
-
-## Architecture (v1)
-
-` ``
-????????????????????????????????????????????????????????????????
-? Manager process (asyncio supervisor) ?
...
</persisted-output>

```

6. user (2026-07-07T03:35:33.145Z)

```

1      ===== DIFF: docs/plan.md =====
2      diff --git a/docs/plan.md b/docs/plan.md
3      index 07b330c..9c0c1f1 100644
4      --- a/docs/plan.md
5      +++ b/docs/plan.md
6      @@ -1,227 +1,113 @@
7      -# Plan: Telegram Video Filter Userbot (v1)
8      -
9      -## TL;DR
10     -A Telethon-based Telegram **userbot** that logs in as the user's own account, watches
11     -new video messages across all channels/groups the account is a member of, applies
12     -user-configured filters (duration, resolution, file size), and **forwards matching
13     -videos to the user's Saved Messages** in real time. Python + Telethon + SQLite +
14     -Docker, multi-user capable (<100), with a docs folder.
15     -
16     -v1 = lean. v2 = digest delivery, on-demand `/search`, public-join + manual-add
17     -ingestion paths, more filters (codec/keyword/mime).
18     -
19     -## Decisions (locked)
20     -- **Stack**:: Python 3.11+ · Telethon (MTProto userbot) · SQLite (aiosqlite) · Docker
21     -- **Ingestion (v1)**:: Userbot only ? Telethon reads all of the user's channel
22     -- **Delivery (v1)**:: Real-time forward of each qualifying video to the user's Saved
23     -- **Filters (v1)**:: duration (min/max seconds), resolution (height==1080 etc.), file
24     -- **Probing**:: Trust Telegram message metadata first (`message.video` +
25     -- **Scale**:: Multi-user, small (<100). Per-user Telethon session, per-user filter
26     -- **Tooling**:: git, `.gitignore`, README, Dockerfile + docker-compose, pytest scaffold,
27     -- **Phased**:: v1 lean now; v2 defers extra ingestion paths, digest, search, more
28     filters.
29     -## Architecture (v1)
30     -
31     -` ``
32     -????????????????????????????????????????????????????????????????
33     -? Manager process (asyncio supervisor) ?
34     -? ?????????????????????????????? ?????????????????????????? ?
35     -? ? Telethon client (per ? ? Filter engine ? ?

```

```

36     -? ? user) ? events.NewMessage? ? (pluggable rules) ? ?
37     -? ? on incoming videos ??? duration/res/size ? ?
38     -? ????????????????????????????????????????????????????????????? ?
39     -? ? metadata ? match ?
40     -? ?????????????????????????????????????????????????????????????
41     -? ? SQLite (aiosqlite): ??
42     -? ? users, channels, videos(seen-state), filter_config ??
43     -? ?????????????????????????????????????????????????????????????
44     -? ? on match ?
45     -? client.forward_messages('me', msg) ? Saved Messages ?
46     -????????????????????????????????????????????????????????????
47     +# Implementation Plan and Status
48     +
49     +This file is the project status ledger. The architecture reference is
50     +[design.md](./design.md); operational procedures are in [runbook.md](./runbook.md).
51     +
52     +## Current State
53     +
54     +The main Telegram userbot workflow is implemented:
55     +
56     +1. Bootstrap Telegram user sessions with `python -m tg_video_filter.bootstrap`.
57     +2. Run one Telethon client per enabled session with `python -m tg_video_filter`.
58     +3. Discover provider-neutral `sources`.
59     +4. Opt in live ingestion with `/watch`.
60     +5. Deduplicate content into `content_items`.
61     +6. Evaluate metadata filters and ADR 0012 policy stages.
62     +7. Persist policy decisions and delivery attempts.
63     +8. Deliver accepted content by forward, link, or both.
64     +9. Explain decisions with `/why <content_item_id>`.
65     +
66     +## Completed Tracks
67     +
68     +| Area | Status | Notes |
69     +|---|---|---|
70     +| Core scaffold | Complete | Python package, Docker, settings, tests, lint, coverage
gate. |
71     +| Data layer | Complete | SQLite schema, migrations through `user_version=5`, provider-
neutral repos and legacy shims. |
72     +| Filter engine | Complete | Duration, resolution, size, GIF, round-video, and nosound
filters. |
73     +| Telethon ingestion | Complete | Metadata extraction, source gate, dedup, policy
evaluation, persistence, delivery. |
74     +| Delivery | Complete | `forward`, `link`, `both`; configurable target; delivery
attempts. |
75     +| Bootstrap | Complete | Interactive login creates session file and default
account/session/config rows. |
76     +| Backfill | Complete | `/backfill [N][h|d|w]` plus `/stop`; broad historical scan. |
77     +| Inline commands | Complete | `/show`, `/set`, source watch commands, `/why`. |
78     +| Content dedup | Complete | Message instance, Telegram `document.id`, metadata
fingerprint. |
79     +| Selective scanning | Complete | `sources.watched` and Saved Messages watchlist
commands. |
80     +| Identity/session split | Complete | `accounts` and `telegram_sessions`, with legacy
`users` shim. |
81     +| Provider-neutral schema | Complete | `sources`, `content_items`, `policies`,
`destinations`, `routing_rules`, `delivery_attempts`. |
82     +| Policy pipeline | Complete | ADR 0012 F1-F7: engine, topics, caption safety, URL
reputation, deep-inspection boundary, audit table, `/why`. |
83     +
84     +## ADR 0012 Phase Closure
85     +
86     +| Phase | Issue | PR | Result |
87     +|---|---|---|---|
88     +| F1 - Policy engine prototype + config persistence | #43 | #50 | Merged |
89     +| F2 - Topic preferences + caption capture | #44 | #51 | Merged |

```

```

90  +| F3 - Caption safety stage | #45 | #52 | Merged |
91  +| F4 - URL/domain reputation stage | #46 | #53 | Merged |
92  +| F5 - Deep inspection fetcher + budget boundary | #47 | #54 | Merged |
93  +| F6 - Policy decisions audit table | #48 | #55 | Merged |
94  +| F7 - `/why` explainability command | #49 | #57 | Merged |
95  +
96  +Umbrella issue #42 is complete. Policy-decision retention cleanup remains open
97  +as follow-up #56.
98  +
99  +## Current Operational Commands
100 +
101 +Config:
102 +
103 +- `/show config`
104 +- `/show commands`
105 +- `/set size <N><unit>`
106 +- `/set length <N><unit>` or `/set max length <N><unit>`
107 +- `/set min length <N><unit>`
108 +- `/set min resolution <N>p` or `/set min res <N>p`
109 +- `/set exclude gif|round|nosound on|off`
110 +
111 +Source scanning:
112 +
113 +- `/channels`
114 +- `/watch <id>|@name|title-fragment>`
115 +- `/unwatch <id>|@name|title-fragment>`
116 +- `/watchlist`
117 +
118 +Backfill and explainability:
119 +
120 +- `/backfill [N][h|d|w]`
121 +- `/stop`
122 +- `/why <content_item_id>`
123 +
124 +## Deferred Work
125 +
126 +- Bot API front door and onboarding commands from ADR 0008.
127 +- Public-channel auto-join/manual-add ingestion.
128 +- Scheduled digest delivery.
129 +- On-demand search/list commands.
130 +- Production deep-inspection fetcher/downloader.
131 +- Policy-decision retention cleanup (#56).
132 +- Horizontal scaling or multi-host coordination.
133 +
134 +## Verification Baseline
135 +
136 +Use the repo-local virtualenv on Windows:
137 +
138 +```powershell
139 +.\.venv\Scripts\python.exe -m ruff check src tests
140 +.\.venv\Scripts\python.exe -m pytest
141 +```
142
143 -## Project structure
144 -```
145 -??? README.md
146 -??? .gitignore
147 -??? .env.example           # TG_API_ID, TG_API_HASH, DB_PATH, ADMIN_USER_IDS
148 -??? pyproject.toml         # deps: telethon,aiosqlite,pydantic,pydantic-
settings
149 -??? .pre-commit-config.yaml # ruff, ruff-format
150 -??? docker-compose.yml     # service: app (mounts ./data for sessions+db)
151 -??? Dockerfile             # python:3.11-slim, install deps, run manager
152 -??? docs/
153 -?   ??? plan.md            # this file

```

```

154 -? ??? design.md # architecture detail
155 -? ??? adr/ # Architecture Decision Records
156 -? ? ??? 0001-userbot-over-botapi.md
157 -? ? ??? 0002-sqlite-over-postgres.md
158 -? ? ??? 0003-metadata-first-probing.md
159 -? ??? runbook.md # deploy, rotate session, handle FloodWait
160 -??? src/tg_video_filter/
161 -? ??? __init__.py
162 -? ??? __main__.py # `python -m tg_video_filter` ? run manager
163 -? ??? config.py # pydantic-settings, env loading
164 -? ??? db.py # aiosqlite schema + migrations + repos
165 -? ??? models.py # User, Channel, Video, FilterConfig pydantic models
166 -? ??? filters/
167 -? ? ??? __init__.py # FilterEngine registry
168 -? ? ??? base.py # Filter ABC: match(metadata) -> bool, reason str
169 -? ? ??? duration.py
170 -? ? ??? resolution.py
171 -? ? ??? size.py
172 -? ??? telethon_client.py # per-user client factory, event wiring
173 -? ??? manager.py # asyncio supervisor: start/stop per-user clients
174 -? ??? delivery.py # forward-to-saved-messages (v1); digest/search in v2
175 -??? data/ # gitignored: sessions/*.session, app.db
176 -??? tests/
177 - ??? conftest.py
178 - ??? test_filters_duration.py
179 - ??? test_filters_resolution.py
180 - ??? test_filters_size.py
181 - ??? test_filter_engine.py
182 -```
183 +The suite currently has broad unit coverage around pure logic and repository
184 +behavior. Live Telethon glue and interactive prompts are intentionally marked
185 +`pragma: no cover`.
186 +
187 +## Original v1 Scope, Now Superseded
188 +
189 +The original v1 plan was a lean userbot that watched every subscribed channel
190 +and forwarded matching videos to Saved Messages. That was useful early, but the
191 +current implementation has moved beyond it:
192 +
193 +- live ingestion is opt-in per source;
194 +- provider-neutral schema is the canonical model;
195 +- delivery is per target/mode with delivery attempts;
196 +- policy decisions are explainable and persisted.
197
198 -## Steps (phased)
199 -
200 -### Phase 0 ? Scaffolding (no runtime logic) ? DONE (PR #2)
201 -1. ? `git init`, create `.gitignore` (Python, `data/`, `*.session`, `.env`),
202 `README.md`.
203 -2. ? `pyproject.toml` with deps: `telethon`, `aiosqlite`, `pydantic`, `pydantic-
204 settings`, `python-dotenv`, `structlog`; dev deps: `pytest`, `pytest-asyncio`, `pytest-cov`,
205 `ruff`, `pre-commit`. Coverage floor `fail_under = 80`.
206 -3. ? `.pre-commit-config.yaml` ? ruff check + format (+ pre-commit-hooks hygiene).
207 -4. ? `Dockerfile` (python:3.11-slim base) + `docker-compose.yml` (mount `./data`).
208 -5. ? `.env.example` documenting `TG_API_ID`, `TG_API_HASH`, `DB_PATH`, `SESSIONS_DIR`,
209 `ADMIN_USER_IDS`, `LOG_LEVEL`.
210 -6. ? `src/tg_video_filter/__init__.py`, `config.py` (pydantic-settings + `NoDecode` CSV
211 parsing), `__main__.py` placeholder; `tests/conftest.py`, `tests/test_config.py`,
212 `tests/test_main.py`.
213 - - Verification: `pytest` 3 passed @ **94.6% coverage**; `ruff check .` + `ruff
214 format --check .` clean; `python -m tg_video_filter` runs end-to-end.
215 - - Note: pydantic-settings v2.14 requires `Annotated[list[int], NoDecode]` to keep
216 CSV env parsing (`ADMIN_USER_IDS=1,2,3`) out of the JSON decoder.
217
218 -### Phase 1 ? Core data layer ? DONE (PR #3)

```

```

211     -7. ? `src/tg_video_filter/config.py` ? done in Phase 0 (pydantic-settings `Settings`).
212     -8. ? `src/tg_video_filter/models.py` ? pydantic models: `User`, `Channel`,
`VideoRecord`, `FilterConfig` (+ `DEFAULT_FILTER_CONFIG`, `dedup_key` helper). Frozen models;
nullable filter bounds to disable a rule.
213     -9. ? `src/tg_video_filter/db.py` ? aiosqlite schema DDL (users, channels, videos,
filter_config), `init_db()`, repository functions (`upsert_user`, `get_user`,
`list_enabled_users`, `upsert_channel`, `get_video`, `insert_video`, `mark_forwarded`,
`load_filter_config`, `save_filter_config`, `list_videos_for_user`). Connection injected so
tests use in-memory SQLite.
214     -10. ? `tests/test_db.py` ? 16 round-trip tests incl. dedup-key IntegrityError, None-
bounds JSON round trip, enabled-user filter, DESC ordering.
215     -   - Verification: `pytest` 19 passed @ **98% coverage**; `ruff check .` + `ruff
format --check .` clean.
216     -
217     -### Phase 2 ? Filter engine (pure, fully unit-testable) ? DONE (PR #4)
218     -11. ? `filters/base.py` ? `VideoMeta` (pydantic), `FilterResult` (dataclass with
`pass_`/`fail` helpers), `Filter` ABC.
219     -12. ? `filters/duration.py`, `resolution.py`, `size.py` ? concrete filters. Each treats
an unknown metadata field as a non-match with a descriptive reason; a `None` bound disables the
rule.
220     -13. ? `filters/__init__.py` ? `FilterEngine`: builds the rule list from `FilterConfig`
(omitting disabled rules), evaluates a `VideoMeta` against all rules with AND semantics,
collects every reason.
221     -   - Verification: `pytest` 58 passed @ **99% coverage** (every filter module 100%);
`ruff check .` + `ruff format --check .` clean.
222     -   - 36 filter tests: table-driven boundary cases (inclusive bounds, None-bounds,
unknown fields), AND-semantics aggregation, partial-config scenarios, vacuous-match guard.
223     -
224     -### Phase 3 ? Telethon ingestion ? DONE (PR #5)
225     -13. ? `telethon_client.py` ? `build_client(user, settings)` per-user Telethon client
factory (file-backed SQLiteSession at `data/sessions/user_{id}.session`);
`make_new_message_handler(...)` wires `events.NewMessage` ? dedup ? filter ? insert ? forward.
226     -14. ? `extract_video_meta(message)` ? pure metadata extraction: inspects
`message.media` (a `Document`) directly for `size`/`mime_type`, and `DocumentAttributeVideo` for
`duration` (float?rounded int)/`w`/`h`. Returns `None` for non-videos. Returns `VideoMeta` with
`None` fields for partial data. (Note: uses `message.media`, not the `.video` convenience
property, which is only present on Telethon's runtime custom Message wrapper ? not the raw TL
`Message`.)
227     -15. ? Dedup: handler queries `db.get_video(user, channel, msg)`; skips if seen.
`_peer_id` collapses `PeerChannel`/`PeerChat`/`PeerUser` to an int.
228     -16. ? `delivery.forward_to_saved` pulled forward (was Phase 4) since the handler
imports it: wraps `client.forward_messages('me', msg)` with `FloodWaitError` handling (sleep
`seconds` + jitter, retry once), marks `forwarded=1` on success.
229     -   - Verification: `pytest` 77 passed @ **89% coverage** (`delivery.py` 100%,
`filters/*` 100%, `db.py` 99%, `models.py` 100%); `ruff check .` + `ruff format --check .`
clean.
230     -   - 19 new tests: 14 for `extract_video_meta` (full metadata, float rounding,
zero?None normalization, non-video/photo/text?None, non-video-attr Document?None, image-size
attr ignored) + `_peer_id` helper; 5 for `forward_to_saved` (success, FloodWait?retry success,
FloodWait?retry fail, generic exception, seconds-attr guard).
231     -   - Uncovered (57%): `telethon_client.py` handler/factory glue ? async Telethon
plumbing requiring a live connection; documented as intentional.
232     -
233     -### Phase 4 ? Delivery + manager ? DONE (PR #6)
234     -17. ? `delivery.py` ? done in Phase 3 (`forward_to_saved` + `FloodWaitError` handling,
100% coverage).
235     -18. ? `manager.py` ? `run(settings, client_runner)` asyncio supervisor: opens one
shared DB connection, loads enabled users, spawns one `asyncio.Task` per user,
`gather(return_exceptions=True)` so one crash doesn't tear down others, installs SIGINT/SIGTERM
handlers. `default_run_user` (the production runner) builds the Telethon client, checks
`is_user_authorized()`, builds the `FilterEngine`, registers `events.NewMessage`, and
`run_until_disconnected()`. Client runner is injected so the supervisor logic is unit-
testable without a live connection.
236     -19. ? `__main__.py` ? real entrypoint: load Settings, configure logging,
`asyncio.run(run(settings, functools.partial(default_run_user, settings=settings)))`, catch

```

```

`KeyboardInterrupt`.
237 - - Verification: `pytest` 82 passed @ **84% coverage** (`manager.py` 67%,
`__main__.py` 85%); `ruff check .` + `ruff format --check .` clean; `python -m tg_video_filter`
runs end-to-end (logs `manager_started users=0`, exits 0).
238 - - 6 new tests: no-users?0, N-users?N-tasks, one-crash-isolated, cancellation-
propagates, entrypoint importable + wires production runner.
239 - - **Uncovered**: `default_run_user` + signal handlers (need a live Telegram session
/ POSIX signals; intentional).
240 -
241 -### Phase 5 ? First-run / onboarding ? DONE (PR #7)
242 -20. ? `src/tg_video_filter/bootstrap.py` ? interactive CLI (`python -m
tg_video_filter.bootstrap`): prompts phone/code/2FA via injectable callables, runs Telethon
`client.start()`, reads `get_me().id`, renames the session file to `user_{id}.session`, then
`persist_user()` inserts the `users` row + default `FilterConfig`. The DB persistence
(`persist_user`) is separated from the interactive login so it's unit-tested.
243 -21. ? Documented in `docs/runbook.md` ($3 onboard, plus cryptg note for prod crypto
perf).
244 - - Verification: `pytest` 86 passed @ **78% coverage** (`bootstrap.persist_user`
covered; `bootstrap()`/`main()` interactive glue intentionally uncovered); `ruff check .` +
`ruff format --check .` clean; `python -m tg_video_filter.bootstrap` runs and reaches the login
prompt.
245 - - Coverage gate temporarily lowered to `fail_under = 70` for the development phase
(interactive entrypoints can't be unit-tested); restores to 80 in Phase 6.
246 -
247 -### Phase 6 ? Tests + lint gate (final polish) ? DONE (PR #8)
248 -22. ? Marked all interactive / Telethon-glue entrypoints `# pragma: no cover`:
249 - - `bootstrap.py`: `bootstrap()`, `main()`, the three `_default*_prompt` stdin
helpers.
250 - - `__main__.py`: `_configure_logging()`, `main()`.
251 - - `manager.py`: `default_run_user()`, `_install_signal_handlers()`.
252 - - `telethon_client.py`: `build_client()`, `on_new_message()`, `_channel_title()`.
253 -23. ? Restored coverage gate `fail_under = 80` (was temporarily 70 during Phases 5?6).
254 -24. ? Final verification gate green; README marked v1 complete.
255 - - **Final coverage: 98.6%** (every module 93%+; `filters/*`, `db.py`, `models.py`,
`delivery.py`, `__main__.py`, `manager.py`, `bootstrap.py` all 100%).
256 -
257 ----
258 -
259 -## v1 sign-off ?
260 -
261 -All 8 PRs merged to `main`. The application is end-to-end usable:
262 -
263 -1. `cp .env.example .env` ? fill `TG_API_ID` / `TG_API_HASH` from my.telegram.org.
264 -2. `docker compose up -d --build` (or run from source).
265 -3. `python -m tg_video_filter.bootstrap` ? interactive login, creates session + user
row.
266 -4. Restart ? manager watches all the account's channels, filters videos by
duration/resolution/size, forwards matches to Saved Messages.
267 -
268 -| Metric | Value |
269 -|---|---|
270 -| Tests | 86 passed |
271 -| Coverage | 98.6% (gate 80%) |
272 -| Lint | ruff clean |
273 -| Entry points verified | `python -m tg_video_filter` + `python -m
tg_video_filter.bootstrap` |
274 -
275 -### Deferred to v2
276 -- Public-channel `/add @channel` join, manual-add ingestion path.
277 -- Scheduled digest delivery + on-demand `/list` `/search`.
278 -- ffmpegprobe `/inspect` deep inspection.
279 -- Codec / keyword / MIME filters.
280 -
281 ----
282 -

```



```

283     -## v1.1 ? Link-share delivery mode (in progress)
284     -
285     -### Phase 2a/2b ? DeliveryMode + link message + dispatcher ? DONE (PR #10)
286     -- **models.py**: added `DeliveryMode` enum (`FORWARD` | `LINK` | `BOTH`) and
`delivery_mode` field to `FilterConfig`. Backwards-compatible (old configs default to
`FORWARD`).
287     -- **delivery.py**: refactored into a `deliver(client, msg, conn, video_id, mode,
link_text=...)` dispatcher. Added pure `build_link_text(...)` for the metadata-summary format
(`? 1080p · 5min · 480MB / #Channel / t.me link`). LINK mode with no link falls back to FORWARD
so no match is dropped.
288     -- **telethon_client.py**: handler now calls `deliver()` with the user's configured
mode, building link text from `message.link` + extracted metadata.
289     -- **manager.py**: passes `cfg.delivery_mode` through to the handler.
290     -- **Tests**: 104 passed @ 97.2% coverage; ruff clean. New tests: `build_link_text`
formatting (full/none/partial metadata), duration+size formatters (parametrized), `deliver`
dispatcher (forward/link/both/fallback/flood-retry/db-marking), FilterConfig delivery_mode JSON
round-trip, old-config-without-field defaults to FORWARD.
291     -- **Docs**: ADR 0004 (pluggable delivery modes), design.md §4 delivery contract,
runbook.md delivery-mode switching instructions.
292     -
293     -### Phase 2c?2e ? Backfill (planned, not started)
294     -### Phase 2c?2e ? Backfill engine + /backfill command ? DONE (PR #12)
295     -- **backfill.py**: `backfill_user()` scans all channel dialogs via
`client.iter_messages(offset_date=cutoff, wait_time=0.5s)` for the last `days` of videos. Reuses
`extract_video_meta` ? `videos` dedup table ? `FilterEngine` ? `deliver()` ? same pipeline as
the live watcher, so backfill + live = no duplicates. FloodWaitError caught and slept on.
Progress callbacks send status messages to Saved Messages.
296     -- **/backfill` command**: `make_backfill_command_handler()` registered on the `me`
chat in `default_run_user`. Parses `/backfill [N][hdw]` (default 90 days, optional hour/week
units). Runs `backfill_user` in a background `asyncio.create_task` so the handler returns
immediately. Sends start/progress/completion messages to `me`.
297     -- **manager.py**: registers the command handler as a second
`events.NewMessage(chats=["me"])` in `default_run_user`, alongside the existing video watcher.
298     -- **Tests**: 127 passed @ 97.8% coverage. 15 new tests: backfill regex parsing (13
parametrized including non-matching inputs), exports verification. `backfill_user` + command
handler marked `# pragma: no cover` (Telethon glue).
299     -- **pyproject.toml**: added per-file-ignore for RUF006 on backfill.py.
300     -- **Docs**: runbook.md updated with /backfill usage (h/w/d units).
301     -
302     -### Phase 2e ? Docs/runbook update ? DONE (PR #12, included)
303     -
304     -## v1.2 ? Per-user delivery target ? DONE
305     -
306     -### Phase 2f ? Configurable delivery destination (ADR 0005)
307     -- **models.py**: added `delivery_target: str = "me"` to `FilterConfig`. Accepts
`"me"` (Saved Messages, default), `"@username"`, or a numeric chat id string. A
`field_validator` coerces int chat ids to str for a uniform JSON blob. `min_length=1` rejects
empty strings. Backwards-compatible (old configs default to `"me"`).
308     -- **delivery.py**: renamed `forward_to_saved` ? `forward_to_target`, added `target:
str = "me"` to `forward_to_target`, `send_link_message`, and `deliver()`. The two hardcoded
`"me"` literals are gone ? delivery destination is now fully config-driven.
309     -- **telethon_client.py** + **backfill.py**: `make_new_message_handler`,
`backfill_user`, `make_backfill_command_handler`, `_run_backfill_and_report` all take
`delivery_target` and thread it to `deliver()`. **Backfill progress messages still go to
`"me"*** (operational, not curated content).
310     -- **manager.py**: passes `cfg.delivery_target` through to both the live handler and
the backfill command handler.
311     -- **Tests**: 146 passed @ 97.2% coverage; ruff clean. New tests: `deliver()` routes
forward/link/both to the configured target (default `"me"`, `"@username"`, numeric id), FakeClient
records target args, `delivery_target` JSON round-trip, old-config-without-field defaults to
`"me"`, int-coercion + empty-string rejection.
312     -- **Docs**: ADR 0005, design.md diagram + control-flow step 8, runbook.md "Routing
matches to a channel you own" section, README feature line.
313     -
314     -## v1.3 ? Content-based deduplication ? DONE

```

```

315 -
316 -### Phase 3a ? Two-tier content dedup + schema migration (ADR 0006)
317 -- **models.py**: added nullable `document_id: int | None` to `VideoRecord` + a
`content_fingerprint` property `(size_bytes, duration_s, height, width)` returning `None` if any
field is missing.
318 -- **db.py**: added `document_id` column, partial unique index
`idx_videos_user_document` (Tier 1, DB-enforced), and `get_video_by_document_id()` /
`get_video_by_content_fingerprint()` queries. **Introduced a `PRAGMA user_version` migration
ladder**: migration 001 idempotently adds the `document_id` column + index to legacy v1
databases. The base `_SCHEMA` no longer creates the index (it references a column that may not
exist on legacy DBs); the migration owns it for both fresh and legacy DBs.
319 -- **telethon_client.py**: new `_extract_document_id()` helper; the live
`on_new_message` handler now runs three-tier dedup (Tier 0 instance ? Tier 1 document.id ? Tier
2 fingerprint) cheapest-first.
320 -- **backfill.py**: same three-tier dedup applied to the historical backfill path.
321 -- **Tests**: 162 passed @ 97% coverage; ruff clean. New tests: fresh-DB column/index
presence, `user_version` bumped to latest, legacy-DB migration (column added, data preserved,
`document_id` defaults NULL), idempotent re-run, partial unique index enforces Tier-1 dedup
(rejects dup `document_id`, allows multiple NULLs).
322 -- **Docs**: ADR 0006, design.md data-model + control-flow step 4 + component diagram,
runbook.md dedup section.
323 -
324 -## Verification
325 -1. **Unit tests green**: `pytest` ? filters, engine, db repos. Coverage ? 80% on
`filters/` and `db.py`.
326 -2. **Lint clean**: `ruff check .` and `ruff format --check .` exit 0.
327 -3. **Pre-commit passes** on a sample commit.
328 -4. **Docker build**: `docker compose build` succeeds; container starts and logs
"manager started, N users".
329 -5. **Dry-run ingestion** (manual, against a test channel): post a short (<5s) and a
long video to a channel the test account is in; confirm only the long one is forwarded to Saved
Messages, and both rows exist in `videos` table with correct metadata.
330 -6. **FloodWait path**: unit-test the delivery retry logic with a mocked
`FloodWaitError(seconds=2)`.
331 -7. **Restart resilience**: stop container, restart, confirm sessions still valid (no
re-login prompt) and seen-state preserved.
332 -
333 -## Decisions / scope
334 -- **Included in v1**: userbot ingestion (all subs), real-time forward to Saved
Messages, 3 filters, multi-user, SQLite, Docker, docs, tests, pre-commit.
335 -- **Excluded (v2)**: public-channel join (`/add @channel`), manual-add ingestion path,
scheduled digest delivery, on-demand `/list` `/search`, ffmpeg `/inspect`, codec/keyword/mime
filters.
336 -- **Assumptions**: user provides their own `api_id`/`api_hash` from my.telegram.org;
runs on a single host (no horizontal scaling needed at <100 users); acceptable to log in once
and keep a persistent session file.
337 -
338 -## Further considerations
339 -1. **Multi-user onboarding UX**: v1 bootstraps users via an interactive CLI script.
Acceptable, or do you want a Telegram-based `register` admin command? (Recommend: CLI for v1,
command in v2.)
340 -2. **Saved Messages vs. a dedicated bot-chat delivery target**: ? Resolved in v1.2 (ADR
0005) ? `delivery_target` is now configurable per-user (`"me"`, `@username`, or chat id).
Default remains Saved Messages.
341 -3. **Filter match semantics**: AND (all rules must pass) vs. OR (any rule). (Recommend:
AND by default ? "videos that satisfy ALL my criteria" ? which matches the stated use case of
filtering TO a quality bar.)
342 +Prefer this file, [design.md](./design.md), and [runbook.md](./runbook.md) over
343 +older phase notes when checking current behavior.
344 ===== DIFF: docs/runbook.md =====
345 diff --git a/docs/runbook.md b/docs/runbook.md
346 index b2e3ad7..030c318 100644
347 --- a/docs/runbook.md
348 +++ b/docs/runbook.md
349 @@ -1,73 +1,116 @@

```

```

350 # Runbook: Telegram Video Filter Userbot
351
352 -Operational guide for deploying, onboarding users, and handling common issues.
353 -See [design.md](./design.md) for architecture and [plan.md](./plan.md) for
354 -build status.
355 +Operational guide for deployment, onboarding, source control, filtering,
356 +backfill, explainability, and troubleshooting.
357
358 ## 1. Prerequisites
359
360 - A Telegram account you control.
361 -- `api_id` and `api_hash` from <https://my.telegram.org> ? *API development
362 - tools*. Create an app (any name/desc) to get these.
363 -- Docker + Docker Compose (recommended), **or** Python 3.11+.
364 -- A persistent directory mounted at `/data` (holds the SQLite DB + session
365 - files ? treat as a credential).
366 -- **Optional but recommended**: install `cryptg` (`pip install cryptg`) for
367 - native crypto acceleration. Without it Telethon falls back to slower
368 - pure-Python encryption (you'll see a warning at startup). For Docker, add
369 - `cryptg` to the image.
370 - > **_Note:_** `cryptg` is included as a project dependency by default
371 - > (`pyproject.toml`); you should already have it installed.
372 -
373 -## 2. First-time deploy (Docker)
374 +- `api_id` and `api_hash` from <https://my.telegram.org>.
375 +- Docker + Docker Compose, or Python 3.11+.
376 +- A persistent `data/` directory for SQLite and Telethon session files.
377 +- Optional local virtualenv for scripts/tests:
378 +
379 +```powershell
380 +python -m venv .venv
381 +.\.venv\Scripts\python.exe -m pip install -e .[dev]
382 +```
383 +
384 +`cryptg` is a runtime dependency and should already be installed through the
385 +project package.
386 +
387 +## 2. First Deploy
388
389 ```bash
390 -# 1. clone
391 git clone https://github.com/avidullu/telegram-video-filter.git
392 cd telegram-video-filter
393
394 -# 2. configure env
395 cp .env.example .env
396 -# edit .env: set TG_API_ID, TG_API_HASH, DB_PATH=/data/app.db
397 -
398 -# 3. (optional) pre-create the data volume
399 -mkdir -p data
400 +# edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR
401
402 -# 4. build + run in background
403 docker compose up -d --build
404 docker compose logs -f app
405 ```
406
407 -Expected startup log: `manager_started {users: 0}` until you onboard a user.
408 +Expected first log state: the manager starts with no runnable sessions until a
409 +user is bootstrapped.
410
411 -## 3. Onboard a user (interactive, once per user)
412 +## 3. Onboard a Telegram User Session
413
414 -v1 onboards via a host-side CLI that performs the interactive Telegram login and

```

```

415 -writes a session file into `data/sessions/`.
416 +Run the bootstrap command in the same environment that can write to `data/`:
417
418     ```bash
419 -# from the repo root, inside the same env/container that runs the app
420 python -m tg_video_filter.bootstrap
421 -# ? prints the banner (session dir + db path)
422 -# ? prompts: phone (E.164), then the login code Telegram sends you,
423 -#           then 2FA password if enabled.
424 -# ? creates data/sessions/user_{id}.session
425 -# ? inserts a users row (enabled=1) + default filter_config
426     ```
427
428 -After onboarding, restart the container so the manager picks up the new user:
429 +It prompts for:
430 +
431 +1. phone number,
432 +2. Telegram login code,
433 +3. 2FA password if enabled.
434 +
435 +It then creates:
436 +
437 +- `accounts` row,
438 +- `telegram_sessions` row,
439 +- legacy-compatible `users` view data,
440 +- default policy/config rows,
441 +- `data/sessions/user_{id}.session`.
442 +
443 +Restart the manager after onboarding:
444
445     ```bash
446     docker compose restart app
447 -# ? logs: client_connected {user_id: <id>}
448     ```
449
450 -> ?? **The session file is a credential.** Anyone with `user_{id}.session` can
451 -> act as that Telegram account. Keep `data/` private and backed up.
452 +Session files are credentials. Back up and protect `data/`.
453 +
454 +## 4. Live Source Scanning
455 +
456 +Live ingestion is opt-in per source.
457 +
458 +Unknown sources are lazily discovered as `watched=false`; they are listed but
459 +not ingested until you watch them. Send commands to your own Saved Messages:
460 +
461 +```text
462 +/channels
463 +/watch <id|@name|title-fragment>
464 +/unwatch <id|@name|title-fragment>
465 +/watchlist
466 +```
467 +
468 +Typical flow:
469 +
470 +1. Let new traffic arrive, or run `/backfill` to discover historical dialogs.
471 +2. Run `/channels`.
472 +3. Run `/watch -1001234567890` or `/watch Some Channel`.
473 +4. Confirm with `/watchlist`.
474 +
475 +## 5. Filter Commands
476 +
477 +These commands persist config changes and hot-reload the in-memory filter
478 +engine; no restart is needed for these fields.
479 +

```

```

480 +```text
481 +/show config
482 +/show commands
483 +/set size 10MB
484 +/set length 15m
485 +/set max length 15m
486 +/set min length 5s
487 +/set min resolution 720p
488 +/set min res 720p
489 +/set exclude gif on
490 +/set exclude round off
491 +/set exclude nosound on
492 +```
493 +
494 +Units:
495
496 -## 4. Adjusting filters
497 +- size: `KB`, `MB`, `GB`, `TB`; default is MB.
498 +- duration: `s`, `m`, `h`; default is seconds.
499
500 -Filters are stored per-user as JSON in `filter_config`. Default v1 config:
501 +Default flat filter config for fresh installs:
502
503 ```json
504 {
505 @@ -75,224 +118,219 @@ Filters are stored per-user as JSON in `filter_config`. Default
v1 config:
506     "max_duration_s": 7200,
507     "target_height": 1080,
508     "max_size_mb": 2048,
509 +   "exclude_gif": true,
510 +   "exclude_round": true,
511 +   "exclude_nosound": false,
512     "delivery_mode": "forward",
513     "delivery_target": "me"
514 }
515 ```
516
517 -To change a user's config (no v1 UI yet ? use sqlite3 directly):
518 +## 6. Delivery Mode and Target
519
520 -```bash
521 -sqlite3 data/app.db
522 -# find the user
523 -SELECT id, phone FROM users;
524 -# update (example: allow up to 4h, drop min duration)
525 -UPDATE filter_config
526 -   SET config_json = json('{"min_duration_s":0,"max_duration_s":14400,"target_height":1
080,"max_size_mb":2048,"delivery_mode":"forward"}')
527 - WHERE user_id = <id>;
528 -.quit
529 -docker compose restart app
530 -```
531 +Delivery mode and target are stored in policy/config JSON. There is no
532 +Saved-Messages command for changing them yet, so update deliberately and restart
533 +the manager.
534
535 -### Backfill historical videos (v1.1)
536 +Modes:
537
538 -> ?? Backfill scans **all** channels for historical videos. On large channels
539 -> it can run for minutes and may trigger Telegram rate limits. Progress is
540 -> reported in real time to your Saved Messages.
541 +| Mode | Behavior |
542 +|---|---|

```

```

543     +| `forward` | Forward the Telegram video message. |
544     +| `link` | Send a `t.me` link plus metadata summary; falls back to forward if no link
exists. |
545     +| `both` | Forward and send the link summary. |
546
547     -While the manager is running, send a message to your **Saved Messages**:
548     +Targets:
549
550     +| Target | Meaning |
551     +|---|---|
552     +| `me` | Saved Messages. |
553     +| `@your-feed` | Public channel or chat username. |
554     +| `-1001234567890` | Numeric Telegram chat/channel id. |
555     +
556     +Use the helper script for targets:
557     +
558     +```powershell
559     +.\.env\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
560     +```
561     -/backfill          # scan last 90 days (default)
562     -/backfill 7d       # scan last 7 days
563     -/backfill 8h       # scan last 8 hours
564     -/backfill 2w       # scan last 2 weeks
565     -/backfill 30       # scan last 30 days
566     -```
567
568     -The bot responds:
569     +If Telethon cannot resolve a target, refresh the entity cache:
570     +
571     +```powershell
572     +.\.env\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
573     +```
574     -? Starting backfill of all channels (last 90 days)...
575     -? Scanning #ChannelName (last 90 days)...
576     -   #ChannelName: 142 videos, 8 matched so far...
577     -? #ChannelName: 156 videos, 11 matched
578     -? Backfill complete: 5 channels scanned, 823 videos found, 47 matched
579     +
580     +Then restart:
581     +
582     +```bash
583     +docker compose restart app
584     +```
585
586     -- Videos already seen (by live watcher or prior backfill) are **skipped**.
587     -- Matches are delivered according to your current `delivery_mode`.
588     -- If a channel triggers a flood wait, it sleeps and continues.
589     +Backfill progress messages always go to Saved Messages; delivered matches honor
590     +`delivery_target`.
591
592     -### Tuning filter criteria live (v1.3)
593     +## 7. Backfill
594
595     -While the manager is running, send these commands to **Saved Messages**:
596     +Backfill scans historical channel/group dialogs and can be expensive on large
597     +accounts. It does not use the live `watched` gate; run it intentionally.
598
599     -```
600     -/show config          # display current filter settings
601     -/show commands       # list all available commands
602     -/set size 10MB        # set max file size
603     -/set length 15m       # set max video duration
604     -/set max length 15m   # same as above (alias)
605     -/set min length 5s    # set min video duration
606     -/set exclude gif on  # exclude Telegram GIFs

```

```

607 -/set exclude round on          # exclude circular video messages
608 -/set exclude nosound on       # exclude videos without audio
609 +``text
610 +/backfill
611 +/backfill 8h
612 +/backfill 7d
613 +/backfill 2w
614 +/stop
615 ``
616
617 -All changes persist to the DB and take effect immediately ? no restart needed.
618 -Exclusions are **off by default** for backwards compatibility.
619 +Behavior:
620
621 -### Switching delivery mode (v1.1)
622 +- default window is 90 days;
623 +- dedup skips already-seen content;
624 +- matching content is delivered with current delivery mode/target;
625 +- progress is reported to Saved Messages;
626 +- flood waits are slept through and logged.
627
628 -The ``delivery_mode`` field controls how matched videos are delivered to your
629 -Saved Messages:
630 +## 8. Policy Pipeline
631
632 -| Mode | What you get |
633 -|---|---|
634 -| ``forward`` (default) | The video itself is forwarded (re-uploaded). |
635 -| ``link`` | A text message with a ``t.me`` link + metadata summary (resolution,
duration, size). No re-upload. |
636 -| ``both`` | Forward the video AND send the link message. |
637 +Metadata filters run first. If they pass, ADR 0012 policy stages run in order:
638
639 -Example ? switch to link-only mode:
640 +1. topic preferences,
641 +2. URL/domain reputation,
642 +3. caption safety,
643 +4. deep inspection boundary.
644
645 -```sql
646 -UPDATE filter_config
647 -   SET config_json = json_set(config_json, '$.delivery_mode', 'link')
648 - WHERE user_id = <id>;
649 -```
650 +Flat ``/set`` commands update ``FilterConfig`` fields inside the default policy
651 +without erasing unknown policy keys.
652
653 -Then restart the manager. Old configs without ``delivery_mode`` default to
654 -``forward`` (backwards-compatible).
655 +Policy-stage config currently has no friendly command UI. Edit
656 +``policies.config_json`` carefully when needed. Example:
657
658 -### Routing matches to a channel you own (v1.2)
659 -
660 -By default matches go to your **Saved Messages**. Set ``delivery_target`` to
661 -route them to a channel instead (see ADR 0005):
662 +```sql
663 +UPDATE policies
664 +   SET config_json = json_set(
665 +     config_json,
666 +     '$.topics',
667 +     json('{"include":["ai"],"exclude":["crypto"],"mode":"any"}')
668 +   )
669 + WHERE account_id = <account_id>
670 +   AND is_default = 1;

```

```

671 +```
672
673 -| Value | Meaning |
674 -|---|---|
675 -| ``"me"`` (default) | Saved Messages. |
676 -| ``"@your-feed"`` | A public channel the account is a member of. |
677 -| ``"-1001234567890"`` | A private channel's chat id (the ``-100?`` form). |
678 +Caption and domain checks:
679
680 ```sql
681 -UPDATE filter_config
682 -   SET config_json = json_set(config_json, '$.delivery_target', '@my-movie-feed')
683 - WHERE user_id = <id>;
684 +UPDATE policies
685 +   SET config_json = json_set(
686 +     config_json,
687 +     '$.security_checks',
688 +     json('{
689 +       "blocked_domains": ["bad.example"],
690 +       "unsafe_caption_terms": ["malware"],
691 +       "unsafe_caption_patterns": ["free\\\\s+money"]
692 +     }')
693 +   )
694 + WHERE account_id = <account_id>
695 +   AND is_default = 1;
696 ```
697
698 -Then restart the manager. Requirements:
699 +Keep `deep_inspect` disabled unless a production fetcher/budget is explicitly
700 +wired:
701 +
702 +```json
703 +{ "deep_inspect": false }
704 +```
705
706 -- The userbot account must be a member of the target channel and have
707 - post rights (admin) if posting is restricted.
708 -- Backfill progress messages still go to Saved Messages ? only matched
709 - videos/links honor `delivery_target`.
710 -- Old configs without `delivery_target` default to ``"me"`` (backwards-compatible).
711 +## 9. Explain Decisions
712
713 -Instead of hand-writing SQL, use the helper script (run with the venv python,
714 -not the bare `python` on your PATH ? the project's deps live in .venv):
715 +Use `/why <content_item_id>` from Saved Messages.
716
717 - # Windows (PowerShell)
718 - .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
719 - # macOS / Linux
720 - .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
721 +Find recent content item ids:
722
723 -#### Troubleshooting: "Cannot find any entity corresponding to ..."
724 +```sql
725 +SELECT id, provider, provider_message_id, matched, seen_at
726 + FROM content_items
727 + ORDER BY id DESC
728 + LIMIT 20;
729 +```
730
731 -If delivery fails with `Cannot find any entity corresponding to "-100..."`,
732 -Telethon's session cache has never recorded that channel's `access_hash`.
733 -Resolve and cache it with:
734 +`/why` behavior:
735

```



```

736 - .\\.venv\\Scripts\\python.exe scripts\\resolve_target.py <user_id> <target>
737 +- reads only content owned by the requesting account;
738 +- renders persisted `policy_decisions` traces when present;
739 +- falls back to recorded item status plus current metadata-filter replay when
740 + no policy decision exists;
741 +- escapes untrusted captions/URLs/metadata and caps the reply under Telegram's
742 + message limit.
743
744 -This stops the manager (the session file can't be shared), refreshes the
745 -entity cache via ``get_dialogs()``, verifies the target resolves, and writes
746 -the cached entity back to the session file. Then restart the manager.
747 +The fallback replay uses current config and persisted metadata. It can differ
748 +from the original decision if config changed, and GIF/round/nosound flags are
749 +not persisted for replay.
750
751 -## 4b. How video deduplication works (v1.3)
752 +## 10. Deduplication
753
754 -The userbot will not forward the same video twice thanks to a **three-tier
755 -dedup** pipeline (checked cheapest-first; see ADR 0006):
756 +Dedup is automatic and metadata-first:
757
758 | Tier | Key | Catches |
759 |---|---|---|
760 -| **0** | `(user_id, channel_id, message_id)` | The same message instance (re-edits,
restarts). |
761 -| **1** | `(user_id, document_id)` | The same Telegram file **forwarded** across
channels ? Telegram reuses `document.id`. DB-enforced via a partial unique index. |
762 -| **2** | `(size_bytes, duration_s, height, width)` | A re-upload of the identical file
(new `document.id`, same bytes). Heuristic lookup, no DB constraint. |
763 -
764 -This is fully automatic and adds no bandwidth cost ? it relies only on metadata
765 -already present in the message. A re-encode (different resolution/codecs/size)
766 -is **intentionally not deduped** ? it is a different file and is evaluated
767 -independently by the filter engine.
768 +| 0 | source/message | Same Telegram message instance. |
769 +| 1 | Telegram `document.id` | Same file forwarded across channels. |
770 +| 2 | size/duration/height/width | Re-upload of identical media metadata. |
771
772 -**Database migration**: existing v1 databases are upgraded automatically on the
773 -next `init_db()` call via a `PRAGMA user_version` migration ladder. The
774 -`document_id` column is added with `ALTER TABLE` (nullable; existing rows get
775 -`NULL`) and the partial unique index is created. The migration is idempotent
776 -and safe to re-run. No manual SQL is needed.
777 +Re-encodes are intentionally treated as distinct content.
778
779 -## 5. Rotate a user session
780 +## 11. Backups
781
782 -Use when a session is suspected compromised, a phone number changes, or login
783 -starts failing.
784 +The important state is `data/`:
785
786 ```bash
787 docker compose stop app
788 -rm data/sessions/user_{id}.session          # remove old session
789 -# also kill it from Telegram side: Settings ? Devices ? terminate the session
790 -python -m tg_video_filter.bootstrap         # re-login interactively
791 +tar -czf backup-$(date +%F).tar.gz data/
792 docker compose start app
793 ```
794
795 -## 6. Handle `FloodWaitError`
796 -
797 -Telegram rate-limits automated actions. The delivery layer already:

```

```

798 -1. catches `FloodWaitError`,
799 -2. sleeps `e.seconds + small_jitter`,
800 -3. retries **once**.
801 -
802 -If you still see floods frequently:
803 -- Reduce how many channels the account is in, or
804 -- Temporarily disable the user (`UPDATE users SET enabled=0 WHERE id=?`) and
805 - wait, or
806 -- Tighten filters so fewer videos are forwarded.
807 +Restore by stopping the app, replacing `data/`, and starting again.
808
809 -Persistent abuse-level automation can get the **account** restricted ? this is
810 -the user's responsibility (see ADR 0001).
811 +## 12. Rotate a Session
812
813 -## 7. Backups
814 -
815 -The entire state is the `data/` directory:
816 +Use this when a session is compromised or login starts failing:
817
818 ```bash
819 docker compose stop app
820 -tar -czf backup-$(date +%F).tar.gz data/
821 +rm data/sessions/user_{id}.session
822 +## Also terminate the session from Telegram: Settings -> Devices.
823 +python -m tg_video_filter.bootstrap
824 docker compose start app
825 ```
826
827 -Restore = stop, replace `data/`, start.
828 +## 13. Observability
829
830 -## 8. Observability
831 -
832 -v1 logs JSON Lines to stdout. Tail with:
833 +Logs go to stdout:
834
835 ```bash
836 -docker compose logs -f app | jq .
837 +docker compose logs -f app
838 ```
839
840 -Key events: `manager_started`, `client_connected`, `video_seen`
841 -(`{matched, reasons}`), `video_forwarded`, `flood_wait {seconds}`,
842 -`client_disconnected`.
843 +Useful events:
844 +
845 +- `manager_started`
846 +- `client_connected`
847 +- `video_seen`
848 +- `video_dedup_document_id`
849 +- `video_dedup_fingerprint`
850 +- `flood_wait`
851 +- `config_updated`
852 +- `client_disconnected`
853
854 -There is no v1 metrics endpoint; rely on logs + Docker restart policy
855 -(`restart: unless-stopped` in compose).
856 +There is no metrics endpoint yet.
857
858 -## 9. Troubleshooting
859 +## 14. Troubleshooting
860
861 | Symptom | Likely cause | Fix |
862 |---|---|---|

```

```

863      -| `client_disconnected` immediately on start | session file missing/invalid | re-run
bootstrap ($5) |
864      -| No `video_seen` events at all | account not in any channel posting videos, or filters
too strict | post a test video to a channel; relax filters ($4) |
865      -| Many `flood_wait` logs | forwarding too fast | see $6 |
866      -| `videos.matched=1` but `forwarded=0` | forward failed after retry | check logs for
the error; the row will not auto-retry ? manually forward or re-trigger by deleting the row |
867      -| Container restarts loop | DB locked / bad env | `docker compose logs app`; verify
`.env` and that `data/` is writable |
868      +| Manager starts with zero users | No bootstrapped session | Run `python -m
tg_video_filter.bootstrap`, then restart. |
869      +| No live `video_seen` events | Source is not watched | Run `/channels`, then `/watch
<source>`. |
870      +| `/channels` is empty | No source discovered yet | Wait for new traffic or run
`/backfill` carefully. |
871      +| Delivery target cannot resolve | Telethon session cache lacks entity | Run
`scripts/resolve_target.py`, then restart. |
872      +| Lots of `flood_wait` logs | Too much delivery/backfill traffic | Stop backfill,
reduce watched sources, tighten filters. |
873      +| `/why` says no trace | Item predates F6 or was metadata-filtered before policy stages
| Use fallback output; future decisions should have traces. |
874      +| DB locked/container loop | Shared process or bad volume permissions | Stop duplicate
processes; check `data/` ownership and `.env`. |
875

```

7. assistant (2026-07-07T03:35:36.038Z)

Now I have the full diff for plan.md and runbook.md. Let me verify concrete claims against the code. Let me check the command surface, migration version, event names, scripts, and helper scripts existence.

8. user (2026-07-07T03:35:39.176Z)

```

===SCRIPTS DIR===
resolve_target.py
set_delivery_target.py
===user_version in db===
208:     then runs any pending migrations via the ``user_version`` ladder.
237:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
257:     """Register a migration that brings the DB up to ``user_version == version``."""
331:     """Run every registered migration whose version > current ``user_version``."""
332:     async with conn.execute("PRAGMA user_version") as cur:
340:         await conn.execute(f"PRAGMA user_version = {version}")
===migration count===
243:#     1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
248:_MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
257:     """Register a migration that brings the DB up to ``user_version == version``."""
262:     _MIGRATIONS.append((version, fn))
336:     for version, fn in _MIGRATIONS:
340:         await conn.execute(f"PRAGMA user_version = {version}")

```

9. assistant (2026-07-07T03:35:42.044Z)

The scripts exist. Now let me verify the highest user_version migration number (plan.md claims `user\_version=5`), and check the log event names, plus the command surface.

10. user (2026-07-07T03:35:44.409Z)

```

===migration registrations===
251:def _migration(
350:@_migration(1)
367:@_migration(2)
519:@_migration(3)
682:@_migration(4)

```

```

778:@_migration(5)
===register calls===
351:async def _migration_001_add_document_id(conn: aiomysql.Connection) -> None:
368:async def _migration_002_identity_split(conn: aiomysql.Connection) -> None:
520:async def _migration_003_domain_schema(conn: aiomysql.Connection) -> None:
683:async def _migration_004_destinations_and_routing(conn: aiomysql.Connection) -> None:
779:async def _migration_005_policy_decisions(conn: aiomysql.Connection) -> None:

```

11. assistant (2026-07-07T03:35:47.835Z)

`user\_version=5` is correct. Now let me verify log event names claimed in runbook (`video\_dedup\_document\_id`, `video\_dedup\_fingerprint`, `config\_updated`), and the command surface (`/show commands`, `/set exclude`, `/watch`, `/why`, `/stop`, `/backfill`).

12. user (2026-07-07T03:35:49.806Z)

```

===dedup log events===
src/tg_video_filter/backfill.py:284:                "backfill_video_seen user=%s channel=%s
msg=%s matched=%s reasons=%s",
src/tg_video_filter/backfill.py:329:                log.warning("backfill_flood_wait channel=%s
seconds=%s", title, e.seconds)
src/tg_video_filter/command_service.py:608:                "config_updated user=%s max_size_mb=%s
min_duration_s=%s max_duration_s=%s "
src/tg_video_filter/delivery.py:111:                log.warning("flood_wait video=%s seconds=%s
retrying_after=%.1f", video_id, e.seconds, wait)
src/tg_video_filter/manager.py:73:                log.info("manager_started sessions=%d db=%s",
len(pairs), settings.db_path)
src/tg_video_filter/manager.py:177:                log.info("client_connected account=%s",
session.account_id)
src/tg_video_filter/telethon_client.py:222:                "video_dedup_document_id user=%s
doc_id=%s channel=%s msg=%s "
src/tg_video_filter/telethon_client.py:250:                "video_dedup_fingerprint user=%s
size=%s dur=%s h=%s w=%s "
src/tg_video_filter/telethon_client.py:335:                "video_seen user=%s channel=%s msg=%s
matched=%s reasons=%s",

```

13. assistant (2026-07-07T03:35:53.602Z)

All log events verified. Now let me verify the command surface. The runbook and plan claim `/show config`, `/show commands`, `/set ...`, `/channels`, `/watch`, `/unwatch`, `/watchlist`, `/backfill`, `/stop`, `/why`. Let me check command_service.py and the router.

14. user (2026-07-07T03:35:55.621Z)

```

===command dispatch===
4:same ``/set``, ``/show`` commands work identically through a Bot API
32:_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
33:_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
34:_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
35:_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
36:_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
37:_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
38:_WHY_RE = re.compile(r"^/why(?:\s+(.*))?$", re.IGNORECASE)
39:_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)
40:_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
41:_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
46:_WATCH_RE = re.compile(r"^/watch\s+(.*)$", re.IGNORECASE)
47:_UNWATCH_RE = re.compile(r"^/unwatch\s+(.*)$", re.IGNORECASE)
48:_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
49:_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
142:        "/show config` ? Show current filter settings",
143:        "/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
145:        "/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
146:        "    or `/set max length <N><unit>` ? same as above",

```

```

148:         "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
150:         "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
151:         "    or `/set min res <N>p` ? same as above",
154:         "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
155:         "`/set exclude round on|off` ? Exclude circular video messages",
156:         "`/set exclude nosound on|off` ? Exclude videos without audio\n"
160:         "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
161:         "`/stop` ? Cancel running backfill",
164:         "`/channels` ? List all known sources with watch status",
165:         "`/watch <id|@name>` ? Start watching a source for new videos",
166:         "`/unwatch <id|@name>` ? Stop watching a source",
167:         "`/watchlist` ? Show only watched sources",
170:         "`/why <content_item_id>` ? Explain why an item passed or failed",
173:         "`/show commands` ? Show this help message",
329:         # --- /set size <value><unit> -----
339:         return CommandResult("? Invalid size format. Use e.g. `/set size 10MB`.")
348:         # --- /set length <value><unit> -----
355:         return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
368:         # --- /set min length <value><unit> -----
375:         return CommandResult("? Invalid duration format. Use e.g. `/set min length
5s`.")
388:         # --- /set min resolution <N>p -----
402:         "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
405:         # --- /set exclude <gif|round|nosound> <on|off> -----
426:         "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
427:         "Example: `/set exclude gif on`"
430:         # --- /show config -----
450:         # --- /why <content_item_id> -----
455:         # --- /show commands -----
467:         """"Parse `/watch <@username|channel_id>` and toggle `watched=1`."""
471:         f"? Source `{target}` not found. Use `/channels` to list "
483:         """"Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
487:         f"? Source `{target}` not found. Use `/channels` to list sources."
500:         "channel. Run `/backfill` to discover historical channels, "
516:         "Use `/channels` to see available sources, then `/watch <id>` "
534:         return CommandResult("? Usage: `/why <content_item_id>`)
537:         return CommandResult("? Usage: `/why <content_item_id>`)

```

15. assistant (2026-07-07T03:36:02.090Z)

All commands verified as existing. Now let me verify a few specific claims:

1. Runbook §5 says filter commands "hot-reload the in-memory filter engine; no restart is needed" ? need to verify.
2. Runbook §8 pipeline order: "topic preferences, URL/domain reputation, caption safety, deep inspection boundary" ? matches ground truth.
3. plan.md exclude defaults, and the FilterConfig defaults JSON in runbook §5.
4. The `/backfill [N][h|d|w]` and default 90 days.

Let me check the FilterConfig defaults and the pipeline order in code.

16. user (2026-07-07T03:36:06.471Z)

===FilterConfig defaults===

```

223:     Generalizes the legacy `FilterConfig.delivery_target` field so one account
225:     per-account `delivery_target`; the default destination is the target used
324:     min_duration_s: int | None = Field(default=5, ge=0)
325:     max_duration_s: int | None = Field(default=7200, ge=0)
327:     # Resolution: a video passes if its height >= target_height.
329:     target_height: int | None = Field(default=1080, ge=1)
332:     max_size_mb: int | None = Field(default=2048, ge=1)

```

```

339:     exclude_gif: bool = Field(
343:     exclude_round: bool = Field(
347:     exclude_nosound: bool = Field(
355:     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
364:     delivery_target: str = Field(
370:     @field_validator("delivery_target", mode="before")
372:     def _coerce_delivery_target(cls, v: object) -> str:
387:         """Ensure min_duration_s <= max_duration_s when both are set."""
389:         self.min_duration_s is not None
390:         and self.max_duration_s is not None
391:         and self.min_duration_s > self.max_duration_s
394:         f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
({self.max_duration_s})"
===pipeline order===
src/tg_video_filter/backfill.py:37:     _evaluate_policy_stages,
src/tg_video_filter/backfill.py:251:         policy_passed, policy_results, policy_id
= await _evaluate_policy_stages(
src/tg_video_filter/command_service.py:238:         lines.append("- No stage trace
recorded.")
src/tg_video_filter/command_service.py:240:         for stage in decision.trace:
src/tg_video_filter/command_service.py:241:         stage_name = str(stage.get("stage_name",
"unknown"))
src/tg_video_filter/command_service.py:242:         stage_passed = bool(stage.get("passed"))
src/tg_video_filter/command_service.py:243:         reason =
_safe_markdown_text(str(stage.get("reason", "no reason recorded")), 300)
src/tg_video_filter/command_service.py:244:         icon = "?" if stage_passed else "?"
src/tg_video_filter/command_service.py:245:         lines.append(f"- {icon}
`{_safe_markdown_text(stage_name, 80)}` ? {reason}")
src/tg_video_filter/command_service.py:246:         metadata = stage.get("metadata")
src/tg_video_filter/command_service.py:259:         "No policy-stage decision trace was recorded
for this item.",
src/tg_video_filter/db.py:1358:     nested stage config can coexist.
src/tg_video_filter/db.py:1407:     """Encode a policy stage trace deterministically."""
src/tg_video_filter/models.py:154:     available for future link/article content while URL policy
stages currently
src/tg_video_filter/models.py:192:     future enrichment/security stages grow into the same
`config_json` column.
src/tg_video_filter/policy.py:3:The policy layer is deliberately small and framework-free in F1:
stages receive
src/tg_video_filter/policy.py:5:the first failure. Later Track F phases add concrete stages.
src/tg_video_filter/policy.py:120:     """Fetcher used only by opt-in Tier 4 policy stages."""
src/tg_video_filter/policy.py:129:     """Immutable snapshot available to a policy stage."""
src/tg_video_filter/policy.py:140:     """Outcome of one policy stage."""

```

17. assistant (2026-07-07T03:36:09.510Z)

Defaults confirmed. Let me verify the pipeline stage order in policy.py (`evaluate\_pipeline` / DEFAULT_PIPELINE) and the exclude field defaults.

18. user (2026-07-07T03:36:12.466Z)

```

===exclude defaults===
    exclude_gif: bool = Field(
        default=True,
        description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
    )
    exclude_round: bool = Field(
        default=True,
        description="If True, exclude circular video messages (round_message=True)",
    )
    exclude_nosound: bool = Field(
        default=False,
        description="If True, exclude videos without an audio track (nosound=True)",
    )

```

```

# How matched videos are delivered: forward the video, send a link, or both.
===pipeline stages order===
1: ""Policy pipeline primitives for ADR 0012.
46:  ""Raised when a deep-inspection budget is exhausted.""
56:  ""Bounded execution budget for opt-in deep inspection.""
167:  ""A single stateless gate in the policy pipeline.""
169:  async def evaluate(self, ctx: PolicyContext) -> StageResult:
176:  ""Parsed ADR 0012 topic preference config.""
189:  ""Parsed ADR 0012 caption safety config.""
208:  ""Parsed ADR 0012 URL/domain reputation config.""
226: def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
227:  ""Parse ``topics`` from a raw policy config object.""
228:  raw_topics = config.get("topics")
229:  if raw_topics is None:
231:  if not isinstance(raw_topics, dict):
232:      raise ValueError("topics must be a JSON object")
233:  include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
234:  exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
237:  mode = raw_topics.get("mode", "any")
239:  raise ValueError("topics.mode must be 'any' or 'all'")
247: def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
248:  ""Parse caption safety settings from ``security_checks``.
251:  ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
260:  raw_security.get("unsafe_caption_terms"),
261:  field_name="security_checks.unsafe_caption_terms",
264:  raw_security.get("unsafe_caption_patterns"),
265:  field_name="security_checks.unsafe_caption_patterns",
288: def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
289:  ""Parse URL/domain reputation settings from ``security_checks``.""
317: def load_topic_preferences(policy: Policy) -> TopicPreferences:
318:  ""Load topic preferences from ``policy.config_json``.""
322:  return parse_topic_preferences(decoded)
325: def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
326:  ""Load caption safety settings from ``policy.config_json``.""
330:  return parse_caption_safety_config(decoded)
333: def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
334:  ""Load URL/domain reputation settings from ``policy.config_json``.""
338:  return parse_domain_reputation_config(decoded)
341: def load_deep_inspection_enabled(policy: Policy) -> bool:
342:  ""Return whether Tier 4 deep inspection is explicitly enabled.""
346:  raw_deep_inspect = decoded.get("deep_inspect", False)
347:  if isinstance(raw_deep_inspect, bool):

```

19. assistant (2026-07-07T03:36:15.044Z)

Let me find where the pipeline stages are assembled in order (the actual `\_evaluate\_policy\_stages` function used by telethon_client/backfill).

20. user (2026-07-07T03:36:17.718Z)

```

src/tg_video_filter/backfill.py
src/tg_video_filter/telethon_client.py
===
src/tg_video_filter/telethon_client.py:442:async def _evaluate_policy_stages(

```

21. user (2026-07-07T03:36:20.274Z)

```

442  async def _evaluate_policy_stages(
443      conn: aiosqlite.Connection,
444      account_id: int,
445      source: Source,
446      content_item: ContentItem,
447  ) -> tuple[bool, list[StageResult], int | None]:
448      ""Evaluate ADR 0012 policy stages for a candidate item.""

```

```

449     policy = await db.get_default_policy(conn, account_id)
450     if policy is None:
451         return True, [], None
452     # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453     # user-facing enable command must land with fetcher/budget wiring.
454     passed, results = await evaluate_pipeline(
455         PolicyContext(content_item=content_item, source=source, policy=policy),
456         [
457             TopicPreferenceStage(),
458             UrlReputationStage(),
459             CaptionSafetyStage(),
460             DeepInspectionStage(),
461         ],
462     )
463     return passed, results, policy.id
464
465
466 async def _record_policy_decision(
467     conn: aiosqlite.Connection,
468     *,
469     inserted: VideoRecord,
470     policy_id: int | None,
471     policy_passed: bool,
472     policy_results: list[StageResult],
473 ) -> None:
474     """Persist a policy trace after the content item has been inserted."""
475     if not policy_results or policy_id is None or inserted.id is None:
476         return
477     await db.record_policy_decision(
478         conn,
479         content_item_id=inserted.id,
480         policy_id=policy_id,
481         passed=policy_passed,
482         trace=stage_results_to_trace(policy_results),
483     )
484
485
486 def _peer_id(peer: object) -> int:
487     """Best-effort numeric id for the peer of a message.
488
489     Telegram represents channel/group/user peers as different TL types; we
490     collapse them to a single int for dedup/storage. The exact subclass isn't
491     important ? only that the id is stable per chat.
492     """
493     # Each Peer* type has a field like channel_id / chat_id / user_id.
494     for field in ("channel_id", "chat_id", "user_id"):
495         val = getattr(peer, field, None)
496         if val is not None:
497             return int(val)
498     # Fallback: hash the repr so we never collide with real ids (negative).
499     return -abs(hash(repr(peer))) % (2**31)
500
501
502 def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
503     """Best-effort channel title from the event, if available."""
504     chat = getattr(event, "chat", None)
505     return getattr(chat, "title", None) if chat is not None else None
506
507
508 class _NewMessageCallback:
509     """Structural type annotation for the handler returned by make*_handler."""
510
511     async def __call__(self, event: NewMessage.Event) -> None: ...
512
513

```



```

514     __all__ = [
515         "build_client",
516         "extract_video_meta",
517         "make_new_message_handler",
518         "_content_item_preview",
519         "_evaluate_policy_stages",
520         "_record_policy_decision",
521         "_message_text",
522     ]
523

```

22. assistant (2026-07-07T03:36:25.710Z)

Important finding. The **code order** is: ``TopicPreferenceStage`, `UrlReputationStage`, `CaptionSafetyStage`, `DeepInspectionStage``. This matches the ground truth: `topic-preferences -> url-reputation -> caption-safety -> deep-inspection`.

The **runbook §8** claims the same order: "1. topic preferences, 2. URL/domain reputation, 3. caption safety, 4. deep inspection boundary." That's correct.

Now let me check the runbook §5 claim about hot-reload ("no restart is needed"), and verify the delivery-mode target claim in §6 that says "There is no Saved-Messages command for changing them yet" (delivery mode/target). Let me check whether ``/set`` reloads the engine.

23. user (2026-07-07T03:36:29.396Z)

```

22:from tg_video_filter.filters import FilterEngine, VideoMeta
27:# Key into the shared engine_holder dict (imported by commands.py's adapter).
28:ENGINE_KEY = "engine"
254:     replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
289:     and ``config_changed`` to decide whether to hot-reload the filter engine.
313:         engine_holder["engine"] = FilterEngine(current_cfg)
===who calls command service / applies config===

```

24. assistant (2026-07-07T03:36:33.828Z)

Hot-reload confirmed at line 313 (``engine_holder["engine"] = FilterEngine(current_cfg)``). The runbook §5 claim is accurate.

Now let me verify a few remaining specifics:

- Runbook §6 helper script ``scripts/set_delivery_target.py <account_id> <target>`` ? does the script take `account_id` or `user_id`? The old runbook said ``<user_id>``.
- The runbook §8 SQL example uses ``account_id`` and ``is_default = 1`` ? verify the policies table column is ``is_default``.
- plan.md "migrations through `user_version=5`" ? confirmed.

25. user (2026-07-07T03:36:36.604Z)

```

===set_delivery_target arg name===
6:     .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
8:     .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
31:     if len(sys.argv) != 3:
32:         print("Usage: python scripts/set_delivery_target.py <user_id> <target>",
file=sys.stderr)
36:     user_id = int(sys.argv[1])
37:     target = sys.argv[2].strip()
50:         "WHERE user_id = ?",
51:         (target, user_id),
55:         print(f"ERROR: no filter_config row for user_id={user_id}.", file=sys.stderr)
64:         "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
===resolve_target arg name===
15:     .\\.venv\\Scripts\\python.exe scripts\\resolve_target.py <user_id> <target>
17:     .venv/bin/python scripts/resolve_target.py <user_id> <target>
42:async def _resolve(user_id: int, target: str) -> int:

```

```

49:     session_path = Path(settings.sessions_dir) / f"user_{user_id}"
91:     if len(sys.argv) != 3:
92:         print("Usage: python scripts/resolve_target.py <user_id> <target>", file=sys.stderr)
95:     return asyncio.run(_resolve(int(sys.argv[1]), sys.argv[2].strip()))
===policies table is_default col===
138:     is_default INTEGER NOT NULL DEFAULT 0,
146: ON policies(account_id) WHERE is_default = 1;
633: # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
637:     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
651:     "ON policies(account_id) WHERE is_default = 1"
737:     JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
919:     is_default=bool(row["is_default"]),
1309:     Exactly one policy per account should carry ``is_default=True``; callers
1315:     """INSERT INTO policies (account_id, name, config_json, is_default,
1320:     is_default=excluded.is_default,

```

26. assistant (2026-07-07T03:36:43.445Z)

Found two important discrepancies:

1. `set_delivery_target.py` and `resolve_target.py` take `<user_id>`, not `<account_id>`.
The runbook (new) says `scripts/set_delivery_target.py <account_id> <target>` and `scripts/resolve_target.py <account_id> <target>` (diff lines 559, 572). But the actual scripts' usage strings and argv parsing use `user_id`. This is a factual mismatch? an operator would type the wrong conceptual value. However, is `user_id` == `account_id` in practice? The script operates on the legacy `filter_config` table `WHERE user_id = ?`. Let me check whether `user_id` and `account_id` are the same numeric value (the legacy shim). This matters for severity.

2. `set_delivery_target.py` writes to the legacy `filter_config` table (`WHERE user_id = ?`), but the runbook §6 says delivery mode/target are stored in "policy/config JSON" and the §8 examples edit `policies`. Let me verify whether the legacy `filter_config` still exists / is still the effective store, or whether this script is stale.

27. user (2026-07-07T03:36:46.767Z)

```

===filter_config table still exists?===
86:CREATE TABLE IF NOT EXISTS filter_config (
222: # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
230:     await conn.execute("DROP TABLE IF EXISTS filter_config")
372: data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
378: FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
440: # Rebuild channels / videos / filter_config so their FK constraints point
487:     "filter_config",
489:     CREATE TABLE filter_config (
514: # filter_config tables were already rebuilt above with FKs to accounts(id),
527: ``channels`` / ``videos`` / ``filter_config`` tables and renames the
===save_filter_config target table===
1887:async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1902:async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:

```

28. user (2026-07-07T03:36:49.161Z)

```

1887 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1888     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1889
1890     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1891     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1892     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1893     Invalid ``config_json`` still raises from JSON/Pydantic validation; callers
1894     do not currently treat malformed persisted config as recoverable.
1895     """
1896     policy = await get_default_policy(conn, user_id)
1897     if policy is None:

```

```

1898         return FilterConfig()
1899     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1900
1901
1902     async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1903         """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1904
1905         ADR 0012: future pipeline keys live in the same ``policies.config_json``
1906         object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1907         merge the flat config fields into the existing object so unknown policy
1908         keys survive legacy ``/set`` command updates.
1909         """
1910         existing = await get_default_policy(conn, user_id)
1911         raw_config: dict[str, object] = {}
1912         if existing is not None:
1913             try:
1914                 raw_config = _policy_config_json_to_dict(existing.config_json)
1915             except (json.JSONDecodeError, ValueError):
1916                 log.warning(
1917                     "resetting malformed policy config while saving FilterConfig user=%s "
1918                     "policy_id=%s",
1919                     user_id,
1920                     existing.id,
1921                     exc_info=True,
1922                 )
1923             raw_config = {}
1924         raw_config.update(cfg.model_dump(mode="json"))
1925         await save_policy_config(conn, user_id, raw_config)
1926
1927
1928     # --- bulk helpers (used by the manager on startup) -----
1929
1930
1931     async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1932         conn: aiomysqlite.Connection, user_id: int, limit: int = 100
1933     ) -> list[VideoRecord]:
1934         """Deprecated shim: list the account's content items projected onto VideoRecord."""
1935         conn.row_factory = aiomysqlite.Row
1936         async with conn.execute(
1937             """SELECT ci.*, s.provider_source_id AS src_pid
1938                FROM content_items ci
1939                JOIN sources s ON s.id = ci.source_id
1940                WHERE ci.account_id = ?
1941                ORDER BY ci.id DESC LIMIT ?""",
1942             (user_id, limit),
1943         ) as cur:
1944             rows = await cur.fetchall()
1945         return [
1946             VideoRecord(
1947                 id=r["id"],
1948                 user_id=user_id,
1949                 channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
1950                 message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
1951                 text=r["text"],
1952                 document_id=(
1953                     _safe_int_cast(r["document_id"], "document_id")
1954                     if r["document_id"] is not None
1955                     else None
1956                 ),

```

29. assistant (2026-07-07T03:36:53.094Z)

Key insight: `filter_config` is renamed to `filter_config_deprecated` under ADR 0011. The `save_filter_config`/`load_filter_config` shims now operate on the `policies` table (keyed by

`account\_id`, which the shim calls `user\_id`). But the helper script `set\_delivery\_target.py` still does raw SQL `UPDATE filter\_config ... WHERE user\_id = ?`. Let me verify the script's actual SQL ? it may write to a now-renamed/deprecated table, making it non-functional.

30. user (2026-07-07T03:36:53.596Z)

```
1      """One-shot helper: set delivery_target for a user.
2
3      Usage (run from the repo root with the VENV python)::
4
5          # Windows (PowerShell)
6          .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
7          # macOS / Linux
8          .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
9
10     Examples::
11
12         .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291
-1003957060579
13         .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291 @my-channel
14         .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291 me
15
16     Note: using the bare ``python`` from your PATH will fail with
17     ``ModuleNotFoundError: No module named 'pydantic_settings'`` because the
18     project's dependencies live in ``.venv``, not the system interpreter.
19
20     Reads the DB path from app settings (``Settings.db_path``) so it respects
21     .env / TG_DB_PATH the same way the manager does.
22     """
23
24     from __future__ import annotations
25
26     import sqlite3
27     import sys
28
29
30     def main() -> int:
31         if len(sys.argv) != 3:
32             print("Usage: python scripts/set_delivery_target.py <user_id> <target>",
file=sys.stderr)
33             print("  target: 'me', '@username', or a chat id like -1001234567890",
file=sys.stderr)
34             return 2
35
36         user_id = int(sys.argv[1])
37         target = sys.argv[2].strip()
38
39         # Read db_path from settings so .env / env vars are honoured.
40         sys.path.insert(0, "src")
41         from tg_video_filter.config import get_settings
42
43         db_path = get_settings().db_path
44
45         conn = sqlite3.connect(db_path)
46         try:
47             cur = conn.execute(
48                 "UPDATE filter_config "
49                 "SET config_json = json_set(config_json, '$.delivery_target', ?) "
50                 "WHERE user_id = ?",
51                 (target, user_id),
52             )
53             conn.commit()
54             if cur.rowcount == 0:
55                 print(f"ERROR: no filter_config row for user_id={user_id}.",
file=sys.stderr)
```

```

56         print("Run `python -m tg_video_filter.bootstrap` first.", file=sys.stderr)
57         return 1
58     finally:
59         conn.close()
60
61     # Show the resulting config so you can verify.
62     conn = sqlite3.connect(db_path)
63     row = conn.execute(
64         "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
65     ).fetchone()
66     conn.close()
67
68     print(f"? delivery_target for user {user_id} set to: {target}")
69     print(f"  current config: {row[0] if row else '(none)'}")
70     print("\nRestart the manager for the change to take effect:")
71     print("  python -m tg_video_filter      # or")
72     print("  docker compose restart app")
73     return 0
74
75
76 if __name__ == "__main__":
77     sys.exit(main())
78

```

31. assistant (2026-07-07T03:37:01.537Z)

This is a significant finding. The `set_delivery_target.py` helper script does `UPDATE filter_config ... WHERE user_id = ?`. But `filter_config` has been **renamed** to `filter_config_deprecated` (db.py line 230 drops it, migration renames it) and the effective delivery config now lives in `policies.config_json`. So on a migrated DB (`user_version=5`), this script would either fail (`no such table: filter_config`) or update a dead deprecated table that the manager never reads.

This is a **pre-existing script bug**, NOT introduced by this PR. But the PR's runbook §6 **actively instructs operators to run this script** (`scripts\set_delivery_target.py <account_id> <target>`) as the recommended way to change the delivery target ? "Use the helper script for targets." An operator following the runbook would run a script that silently no-ops or errors against a modern DB.

However ? my review scope is "doc claims in THIS PR diff that are **FACTUALLY WRONG** or **MISLEADING** vs the merged code." The runbook recommending this broken script IS a doc claim. And the arg name mismatch (`<account_id>` vs script's `<user_id>`) is a doc claim in the diff. Let me verify the table is actually renamed on a migrated DB by running the migration, to be precise about whether the script errors or no-ops.

32. user (2026-07-07T03:39:05.773Z)

Command running in background with ID: b8vum7yp0. Output is being written to:
 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b8vum7yp0.output. You will be notified when it completes. To check interim output, use Read on that file path.

33. user (2026-07-07T03:39:13.131Z)

Traceback (most recent call last):

```

File "<string>", line 16, in <module>
  asyncio.run(main())
~~~~~^~~~~~^~~~~~^~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
  return runner.run(main)
      ~~~~~^~~~~~^~~~~~^~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run

```

```

    return self._loop.run_until_complete(task)
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
    ~~~~~^
File "<string>", line 7, in main
    await db.init_db(conn)
File "C:\Users\avidu\Projects\_wt-pr58\src\tg_video_filter\db.py", line 210, in init_db
    async with aiosqlite.connect(db_path) as conn:
    ~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 181,
in __aenter__
    return await self
    ~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 168,
in __connect
    self._connection = await future
    ~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 466,
in connector
    return sqlite3.connect(loc, **kwargs)
    ~~~~~^
sqlite3.OperationalError: unable to open database file

```

34. assistant (2026-07-07T03:39:16.206Z)

`init\_db` takes a path, not a connection. Let me fix the harness.

35. user (2026-07-07T03:39:17.100Z)

204:async def init_db(db_path: str) -> None:

migration; putting it in ``\_SCHEMA`` would break ``executescript`` on legacy DBs that don't yet have the column.

async def init_db(db_path: str) -> None:

"""Open the database at ``db\_path`` and create/migrate the schema.

Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
then runs any pending migrations via the ``user\_version`` ladder.
"""

async with aiosqlite.connect(db_path) as conn:
 await _apply_schema(conn)

async def _apply_schema(conn: aiosqlite.Connection) -> None:

Enable WAL mode (concurrent reads+writes), foreign key enforcement,
and a busy timeout so concurrent access waits instead of failing immediately.
await conn.execute("PRAGMA journal_mode = WAL")
await conn.execute("PRAGMA foreign_keys = ON")
await conn.execute("PRAGMA busy_timeout = 5000")
await conn.executescript(_SCHEMA)
ADR 0011 housekeeping: ``\_SCHEMA`` still creates the legacy
``channels`` / ``videos`` / ``filter\_config`` tables (they're needed for
migration 002's FK rebuild and migration 003's backfill on legacy DBs).
Once migration 003 has run (signalled by ``channels\_deprecated`` existing),
any *empty* recreations of those legacy tables from a subsequent
``init\_db`` are dropped here so they don't shadow the renamed copies.
if await _table_exists(conn, "channels_deprecated"):

```

        await conn.execute("DROP TABLE IF EXISTS channels")
        await conn.execute("DROP TABLE IF EXISTS videos")
        await conn.execute("DROP TABLE IF EXISTS filter_config")
    await _apply_migrations(conn)
    await conn.commit()

```

--- migrations -----

36. assistant (2026-07-07T03:39:19.982Z)

Confirmed: after migration, `filter\_config` is DROPPED (line 230), and migration 002/003 renames it to `filter\_config\_deprecated`. So `filter\_config` does NOT exist on a fully-migrated fresh DB. Let me confirm with a real file-based DB.

37. user (2026-07-07T03:39:27.386Z)

```

filter_config present? False
filter_config_deprecated present? True
policies present? True
user_version: 5

```

38. assistant (2026-07-07T03:39:36.545Z)

Confirmed definitively:

- On a fresh/migrated DB (`user\_version=5`), the table `filter\_config` **does not exist** ? it's `filter\_config\_deprecated`.
- `set\_delivery\_target.py` runs `UPDATE filter\_config ... WHERE user\_id = ?` ? will raise `sqlite3.OperationalError: no such table: filter\_config`.

The runbook §6 (this PR) actively instructs: "Use the helper script for targets: `scripts\set\_delivery\_target.py <account\_id> <target>`". Following this runbook step, the operator's delivery-target change **fails outright**. This is a high-severity operational-runbook error (a deploy/config step that is broken).

Now, is this a pre-existing script bug that the PR merely references, or is the runbook wrong? The doc scope is "doc claims in THIS PR diff." The runbook diff added the instruction to use this script with ``. The claim "Use the helper script for targets" is misleading because the script is broken against the current schema. This is a legitimate high finding for a runbook accuracy review. I'll note the root cause is the stale script but the runbook step is what an operator acts on.

Let me also verify `resolve\_target.py` ? it uses session files and `get\_dialogs`, doesn't touch `filter\_config`, so it may still work. Let me confirm its arg (`user\_id`) and whether it references the session path format correctly.

39. user (2026-07-07T03:39:37.721Z)

```

async def _resolve(user_id: int, target: str) -> int:
    sys.path.insert(0, "src")
    from telethon import TelegramClient

    from tg_video_filter.config import get_settings

    settings = get_settings()
    session_path = Path(settings.sessions_dir) / f"user_{user_id}"
    if not session_path.with_suffix(".session").exists():
        print(f"ERROR: session file not found: {session_path}.session", file=sys.stderr)
        print("Run `python -m tg_video_filter.bootstrap` first.", file=sys.stderr)
        return 1

    client = TelegramClient(str(session_path), settings.tg_api_id, settings.tg_api_hash)
    await client.connect()

```

```

try:
    if not await client.is_user_authorized():
        print("ERROR: session is not authorized. Re-run bootstrap.", file=sys.stderr)
        return 1

```

40. assistant (2026-07-07T03:39:42.140Z)

`resolve\_target.py` uses the session file `user\_{user\_id}.session` ? this doesn't touch the deprecated table, so it works. But its arg is `user\_id`, and the session filename is `user\_{id}.session` where `{id}` is the Telegram user id. The runbook now calls this ``.

Now, is `account\_id` numerically the same as `user\_id`/Telegram id? Let me check the identity split ? `accounts.id` vs the Telegram user id used in session filenames. This determines whether the arg-name mismatch is merely cosmetic or actually wrong-valued.

41. user (2026-07-07T03:39:45.354Z)

```

===accounts table===
CREATE TABLE IF NOT EXISTS accounts (
    id            INTEGER PRIMARY KEY,
    telegram_id   INTEGER UNIQUE,
    provider      TEXT NOT NULL DEFAULT 'telegram',
    display_name  TEXT,
    created_at    TEXT NOT NULL
);
===bootstrap: how account id set + session filename===
9: ``<sessions_dir>/user_{id}.session``, and inserts a ``users`` row with the
48:  # NOTE: Telethon only invokes this if your account presents a 2FA
54:  """Insert/update the (account, session) pair implied by ``user`` + seed config.
58:  ADR 0010: onboarding now writes one row to ``accounts`` (the product
65:      await db.upsert_account(conn, Account(id=user.id, telegram_id=user.id))
69:      account_id=user.id,
71:      session_path=f"user_{user.id}.session",
75:      await db.save_filter_config(conn, user.id, DEFAULT_FILTER_CONFIG)
76:      persisted = await db.get_user(conn, user.id)
93:  4. ``get_me()`` to read the authorized user id.
116:  # account actually presents a 2FA challenge (which happens AFTER phone +
118:  # and wrong for accounts without 2FA.
125:  me = await client.get_me()
126:  user_id: int = me.id # type: ignore[union-attr]
128:  # Telegram's get_me() returns phone in E.164 format (already has "+").
129:  log.info("login_success user_id=%s phone=%s", user_id, phone)
134:  final_path = sessions_dir / f"user_{user_id}.session"
146:  user = User(id=user_id, phone=phone, enabled=True)
148:  log.info("user_persisted user_id=%s db=%s", persisted.id, settings.db_path)
166:  print(f"\nDone. User {user.id} registered with default filters.")

```

42. assistant (2026-07-07T03:39:57.278Z)

Confirmed: `db.upsert\_account(conn, Account(id=user.id, telegram\_id=user.id))` ? the `accounts.id` **equals** the Telegram user id (`user.id`), which is the same value used in the session filename `user\_{user.id}.session`. So `account\_id` == `user\_id` numerically. This means the runbook calling the arg `` while the script says `` is **not** a wrong-value problem ? same integer either way. So the arg-name change is cosmetic/harmless (low, not worth reporting as wrong).

The real, high-severity problem stands: `set\_delivery\_target.py` writes to the non-existent `filter\_config` table. The runbook §6 tells operators to use it. Let me double-check the runbook doesn't hedge this. Re-reading the diff: "Use the helper script for targets: `.\.venv\Scripts\python.exe scripts\set\_delivery\_target.py <account\_id> <target>`" ? no caveat. This is the recommended path. Confirmed high.

Now let me verify remaining plan.md/runbook.md claims that could be wrong:

1. Runbook §6 modes table: `forward`/`link`/`both` ? matches ground truth. Good.
2. Runbook §9 `/why` ? "reads only content owned by requesting account" (account-scoped), matches. "renders persisted policy_decisions traces", "escapes untrusted..." ? matches ground truth.
3. plan.md "Filter engine: Duration, resolution, size, GIF, round-video, and nosound filters" ? matches.
4. plan.md ADR 0012 phase table F1-F7 with issue/PR numbers. Let me spot-check the ADR 0012 doc for the phase mapping and #56 deferral.
5. Runbook §4 "Unknown sources are lazily discovered as `watched=false`" ? matches opt-in ground truth.

Let me verify the ADR 0012 phase/issue/PR numbers and the deep_inspect "drops items" behavior, plus check the design.md and command_service.py help-text edits and pyproject/Dockerfile diffs for accuracy.

43. user (2026-07-07T03:39:59.795Z)

```
=== pyproject diff ===
diff --git a/pyproject.toml b/pyproject.toml
index 14ea202..83c4aad 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -5,7 +5,7 @@ build-backend = "hatchling.build"
[project]
  name = "tg-video-filter"
  version = "0.1.0"
-description = "Telethon userbot that filters Telegram channel videos by duration, resolution,
and size, and forwards matches to Saved Messages."
+description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
and explains policy decisions."
  readme = "README.md"
  requires-python = ">=3.11"
  license = { text = "TBD" }
=== Dockerfile diff ===
diff --git a/Dockerfile b/Dockerfile
index 9f4ffa3..a0c882b 100644
--- a/Dockerfile
+++ b/Dockerfile
@@ -25,6 +25,5 @@ ENV DB_PATH=/data/app.db \
  SESSIONS_DIR=/data/sessions \
  LOG_LEVEL=INFO

-# Run the manager entrypoint (Phase 4). For now this will fail fast until
-# __main__.py lands; that's expected at the Phase 0 stage.
+# Run the manager entrypoint.
  CMD ["python", "-m", "tg_video_filter"]
```

44. assistant (2026-07-07T03:40:03.748Z)

pyproject and Dockerfile diffs are accurate. The Dockerfile env vars (`DB\_PATH`, `SESSIONS\_DIR`) match the runbook's `.env` claim. Let me verify the env var names the runbook and plan use match config.py. The runbook §2 says "edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR". Dockerfile ENV uses `DB\_PATH`/`SESSIONS\_DIR` (no TG_ prefix), but config.py may use a prefix. Let me check.

45. user (2026-07-07T03:40:05.884Z)

```
19: All fields map 1:1 to env vars (case-insensitive). `TG_API_ID` is the
31: tg_api_id: int = Field(..., description="Telegram API id from my.telegram.org")
36: db_path: str = Field(default="data/app.db", description="SQLite database path")
37: sessions_dir: str = Field(
```

46. user (2026-07-07T03:40:09.743Z)

```

"""Application configuration via pydantic-settings.

Settings are loaded from environment variables (and an optional .env file).
See .env.example for documentation of each variable.
"""

from __future__ import annotations

from functools import lru_cache
from typing import Annotated, Literal

from pydantic import Field, field_validator
from pydantic_settings import BaseSettings, NoDecode, SettingsConfigDict

class Settings(BaseSettings):
    """Strongly-typed app settings.

    All fields map 1:1 to env vars (case-insensitive). ``TG_API_ID`` is the
    Telegram application id; ``TG_API_HASH`` its secret. Both come from
    https://my.telegram.org.
    """

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    # Telegram MTPROTO credentials (from my.telegram.org).
    tg_api_id: int = Field(..., description="Telegram API id from my.telegram.org")
    tg_api_hash: str = Field(..., min_length=8, description="Telegram API hash")

    # Storage paths. Defaults assume host runs from repo root; Docker overrides
    # these to /data/... via the Dockerfile/compose environment.
    db_path: str = Field(default="data/app.db", description="SQLite database path")
    sessions_dir: str = Field(
        default="data/sessions", description="Directory for per-user Telethon session files"
    )

    # Admin/whitelist. Comma-separated in env -> parsed to list[int].
    # NoDecode tells pydantic-settings NOT to JSON-parse this field, so the
    # raw "1,2,3" string reaches our field_validator below.
    admin_user_ids: Annotated[list[int], NoDecode] = Field(
        default_factory=list,
        description="Comma-separated Telegram user ids allowed to run admin commands",
    )

    log_level: Literal["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"] = Field(
        default="INFO", description="Root log level"
    )

    @field_validator("admin_user_ids", mode="before")
    @classmethod
    def _split_csv(cls, v: object) -> object:
        """Allow ``ADMIN_USER_IDS=1,2,3`` in env.

        With ``NoDecode``, pydantic-settings hands us the raw env string here.
        Programmatic construction may pass a list[int] directly, which we pass
        through unchanged.
        """

```

47. assistant (2026-07-07T03:40:17.405Z)

```

Env vars map 1:1, case-insensitive: `db_path`?`DB_PATH`, `sessions_dir`?`SESSIONS_DIR`,
`tg_api_id`?`TG_API_ID`. The runbook's `.env` list (`TG_API_ID`, `TG_API_HASH`, `DB_PATH`,

```

SESSIONS_DIR`) is accurate.

Now let me verify the bootstrap flow claim in runbook §3. The runbook says bootstrap creates: `accounts` row, `telegram\_sessions` row, "legacy-compatible `users` view data", "default policy/config rows", session file. From bootstrap.py I saw: `upsert\_account`, session row (`account\_id`, `session\_path`), `save\_filter\_config` (writes to policies default), `get\_user`. There's no literal "users view" ? `get\_user` is a shim. This is close enough / not misleading. Good.

Now the plan.md ADR 0012 phase table. Let me verify F1-F7 issue/PR numbers against the ADR 0012 doc and the deep_inspect drop behavior. Let me also verify the runbook §8 claim "Keep deep_inspect disabled unless a production fetcher/budget is explicitly wired" and that enabling it drops items (ground truth). Let me check the DeepInspectionStage behavior when fetcher is None.

48. user (2026-07-07T03:40:18.896Z)

```
721:class DeepInspectionStage:
class DeepInspectionStage:
    """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""

    stage_name = "deep-inspection"

    async def evaluate(self, ctx: PolicyContext) -> StageResult:
        try:
            enabled = load_deep_inspection_enabled(ctx.policy)
        except (json.JSONDecodeError, ValueError) as exc:
            log.warning(
                "deep inspection disabled due to invalid config policy_id=%s",
                ctx.policy.id,
                exc_info=True,
            )
            return StageResult.pass_(
                self.stage_name,
                f"invalid deep inspection config ignored: {exc}",
                invalid_deep_inspection_config=True,
            )
        if not enabled:
            return StageResult.pass_(self.stage_name, "deep inspection disabled")

        budget_result = _deep_inspection_budget_result(ctx)
        if budget_result is not None:
            return budget_result

        if ctx.fetcher is None:
            return StageResult.fail(
                self.stage_name,
                "deep inspection fetcher missing",
                missing_fetcher=True,
            )

        assert ctx.budget is not None
        try:
            return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
        except PolicyBudgetExceeded as exc:
            return StageResult.fail(
                self.stage_name,
                "deep inspection budget exceeded",
                budget_exceeded=True,
                exceeded_budget=exc.exceeded_budget,
                **exc.metadata,
            )
        except Exception as exc:
            log.warning(
                "deep inspection fetcher failed policy_id=%s",
```

```

        ctx.policy.id,
        exc_info=True,
    )
    return StageResult.fail(
        self.stage_name,
        "deep inspection fetcher failed",
        fetcher_error_type=type(exc).__name__,
        fetcher_error=str(exc),
    )

```

```

def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
    budget = ctx.budget
    if budget is None:
        return StageResult.fail(
            DeepInspectionStage.stage_name,
            "deep inspection budget missing",
            missing_budget=True,
        )
    if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent <= 0:
        return StageResult.fail(
            DeepInspectionStage.stage_name,
            "deep inspection budget invalid",
        )

```

49. assistant (2026-07-07T03:40:23.272Z)

Confirmed the ground truth: with `deep\_inspect: true` and `ctx.budget`/`ctx.fetcher` = None in the live path, `DeepInspectionStage` returns `fail` ("deep inspection budget missing") ? the item is dropped. The runbook §8 correctly warns: "Keep `deep\_inspect` disabled unless a production fetcher/budget is explicitly wired." This is accurate.

Now let me verify the ADR 0012 phase table numbers and check the remaining doc diffs (design.md, README, ADR diffs) briefly for wrong claims. Let me check ADR 0012 diff and the phase mapping.

50. user (2026-07-07T03:40:26.243Z)

```

=== ADR 0012 phase refs in code/adr ===
3:- **Status**: Accepted and implemented (F1-F7 merged)
11:This ADR is implemented through the F1-F7 PR sequence tracked by umbrella
12:issue #42:
14:- #50 ? policy engine prototype and config persistence.
15:- #51 ? topic preferences and caption capture.
16:- #52 ? caption safety.
17:- #53 ? URL/domain reputation.
18:- #54 ? deep-inspection budget boundary.
19:- #55 ? `policy_decisions` audit table.
20:- #57 ? `/why <content_item_id>` explainability command.
23:issue #56.
114:`ContentFetcher` is an implementation detail introduced in F5, not a
115:requirement for metadata-only stages. F1-F4 must work when `ctx.fetcher` is
150:`content_items.text`, F2 includes the ingestion work to capture
221:Until F6 lands, rejected-item traces are in-memory/log-only. F6 introduces
222:the persistent `policy_decisions` table; F7 depends on F6 because `/why`
243:| **F1** ? Policy engine prototype | `PolicyStage`, `evaluate_pipeline()`, config
preservation, live hot-path wiring | #50 | Merged |
244:| **F2** ? Topic preferences | `topics` parsing, source-title/caption matching, caption
capture | #51 | Merged |
245:| **F3** ? Caption safety | Unsafe caption term/pattern scanning | #52 | Merged |
246:| **F4** ? URL/domain reputation | Captured-text URL extraction and domain blacklist checks
with no page fetch | #53 | Merged |
247:| **F5** ? Deep inspection | `ContentFetcher` protocol and budget boundary | #54 | Merged |
248:| **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected
policy-stage decisions | #55 | Merged |
249:| **F7** ? Explainability endpoint | `/why <content_item_id>` command showing stored traces
and safe fallback output | #57 | Merged |

```

259: time; F6 persists that trace for later `/why` lookups.
269: (metadata-only vs. I/O-heavy). Mitigated by keeping F1-F4 fetcher-free and
272: F1 must avoid dropping pipeline keys when legacy config commands rewrite

51. assistant (2026-07-07T03:40:33.737Z)

The plan.md ADR 0012 phase table matches the ADR 0012 doc: F1?#43/#50, F2?#44/#51, F3?#45/#52, F4?#46/#53, F5?#47/#54, F6?#48/#55, F7?#49/#57, umbrella #42, retention follow-up #56. Consistent and accurate.

Wait ? plan.md's phase table maps stage order in the description column: "engine, topics, caption safety, URL reputation, deep-inspection boundary, audit table, /why" (plan.md line 82). Note it lists "caption safety, URL reputation" ? but that's just listing the F-phases in F-number order (F3 caption safety before F4 URL reputation), which is the development order, NOT the runtime pipeline order. The runtime order is topic?url?caption?deep. This is the "Completed Tracks" table summarizing F1-F7 by phase number, not claiming pipeline execution order, so it's not misleading. The runbook §8 explicitly documents runtime order correctly (topic?url?caption?deep). Good.

Let me now verify the runbook §9 SQL for `/why` (`SELECT ... matched ... FROM content\_items`) ? does `content\_items` have a `matched` column and `seen\_at`? And check the design.md/README diffs briefly for any wrong claim.

52. user (2026-07-07T03:40:36.385Z)

```
===content_items columns===
CREATE TABLE IF NOT EXISTS content_items (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id        INTEGER NOT NULL REFERENCES accounts(id),
  source_id         INTEGER NOT NULL REFERENCES sources(id),
  provider          TEXT NOT NULL DEFAULT 'telegram',
  provider_message_id TEXT NOT NULL,
  content_type      TEXT NOT NULL DEFAULT 'video',
  text              TEXT,
  link_url          TEXT,
  media_ref         TEXT,
  document_id       TEXT,
  duration_s        INTEGER,
  width             INTEGER,
  height            INTEGER,
  size_bytes        INTEGER,
  mime_type         TEXT,
  dedup_key         TEXT NOT NULL,
  matched           INTEGER NOT NULL DEFAULT 0,
  seen_at          TEXT NOT NULL,
  UNIQUE (account_id, source_id, provider_message_id),
  UNIQUE (account_id, dedup_key)
);
```

53. assistant (2026-07-07T03:40:39.243Z)

The runbook §9 SQL (`SELECT id, provider, provider\_message\_id, matched, seen\_at FROM content\_items`) ? all columns exist. Accurate.

Now let me quickly scan the design.md and README diffs for any wrong claim, plus the command_service.py/models.py/tests diffs (source doc-accuracy edits).

54. user (2026-07-07T03:40:41.674Z)

```
=== command_service.py diff ===
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 0ab172a..79fef65 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
```

```

@@ -150,11 +150,11 @@ def format_commands() -> str:
    "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
    "    or `/set min res <N>p` ? same as above",
    "",
-    "? **Content exclusions** (all off by default)",
+    "? **Content exclusions**",
    "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
    "`/set exclude round on|off` ? Exclude circular video messages",
    "`/set exclude nosound on|off` ? Exclude videos without audio\n"
-    "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
+    "    Defaults: GIF/round excluded, nosound allowed.",
    "",
    "? **Backfill**",
    "`/backfill [N][hwd]` ? Scan last N hours/days/weeks of channel history (default
90d)",
=== models.py diff ===
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index fd4c85e..5c36b88 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -149,9 +149,10 @@ class Source(BaseModel):
    class ContentItem(BaseModel):
        """A normalized content record discovered from any :class:`Source` (ADR 0011).

-        Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
-        (`content_type='video'`); `text` / `link_url` are landed now but stay
-        NULL until later tracks populate them (avoids a future `ALTER TABLE`).
+        Generalizes the legacy :class:`VideoRecord`. The Telegram path stores videos
+        today (`content_type='video'`) and captures caption text; `link_url` is
+        available for future link/article content while URL policy stages currently
+        extract links from captured text at evaluation time.
+        `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
+        wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
        """

@@ -198,9 +199,7 @@ class Policy(BaseModel):
    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    name: str = Field(description="Policy name; 'default' for the account default policy")
-    config_json: str = Field(
-        description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
-    )
+    config_json: str = Field(description="FilterConfig JSON blob plus ADR 0012 policy keys")
    is_default: bool = Field(
        default=False, description="Exactly one per account (UNIQUE partial index)"
    )

@@ -313,11 +312,12 @@ class PolicyDecision(BaseModel):

    class FilterConfig(BaseModel):
-        """Per-user filter configuration.
+        """Flat per-account filter configuration.

-        v1 rules: duration bounds, minimum height (resolution), and max file size.
-        Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
-        new field later is backwards-compatible.
+        Stored inside the account's default `policies.config_json` object through
+        compatibility shims. Legacy `filter_config` callers still work, but new
+        policy keys can live alongside these flat fields without being erased by
+        `/set` command updates.
        """

        # Duration bounds in seconds. None means unbounded on that side.

```

These source doc-accuracy edits are correct:

- The ``format_commands()`` help text now correctly says "Defaults: GIF/round excluded, nosound allowed" (matches ``exclude_gif=True, exclude_round=True, exclude_nosound=False``). The old "all off by default" was wrong; this fixes it. Good.
- `models.py` docstrings updated accurately.

My task dimension is `plan.md` and `runbook.md` specifically. I've thoroughly verified both. Let me do one final check on the `plan.md` "Verification Baseline" commands (``ruff check src tests`, `pytest``) and `runbook $5`'s ``/set min resolution`` claim ? the `runbook $5` command list omits ``/set min resolution``. Wait, let me recheck. `Runbook $5` lists ``/set min resolution 720p`` and ``/set min res 720p`` (diff lines 487-488). Good, present.

Let me verify the one remaining `runbook` claim I haven't confirmed: `$6` says "There is no `Saved-Messages` command for changing them [delivery mode/target] yet." Confirmed earlier ? no ``/set delivery`` command exists in `command_service.py` (the ``_EXCLUDE_RE``, ``_SIZE_RE`` etc. don't cover delivery). Accurate.

Now let me confirm the final high-severity finding precisely: does the `runbook` still reference ``sqlite3 data/app.db`` with ``SELECT ... FROM users`` anywhere that would break? The new `runbook $9/$8` use ``content_items`/`policies`` (correct). The `set_delivery_target` script is the one broken operational step. Let me also double check: is there any chance the DB in production is NOT migrated (legacy v1 with ``filter_config``)? No ? `plan.md` itself claims "migrations through `user_version=5`" and `init_db` always migrates. Any live DB is at v5, so ``filter_config`` is gone. The finding is solid.

Let me get exact line numbers in the `runbook` file for the finding.

56. user (2026-07-07T03:40:55.328Z)

151:Use the helper script for targets:

154: `.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>`

160: `.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>`

333:| Delivery target cannot resolve | Telethon session cache lacks entity | Run ``scripts/resolve_target.py``, then restart. |

57. assistant (2026-07-07T03:40:58.583Z)

I have the line numbers. Let me do a final verification that the script actually errors (not silently no-ops) by running it against the migrated test DB I created, to nail down the exact failure mode for the finding.

58. user (2026-07-07T03:41:03.825Z)

ERROR TYPE: `OperationalError`

ERROR MSG: `no such table: filter_config`

59. assistant (2026-07-07T03:41:14.872Z)

Confirmed definitively: ``set_delivery_target.py``'s SQL (``UPDATE filter_config ... WHERE user_id = ?``) raises ``sqlite3.OperationalError: no such table: filter_config`` on any migrated (``user_version=5``) database ? which is every live database. The `runbook $6` recommends this script as the way to change the delivery target, so an operator following it hits a hard error and cannot change their delivery target via the documented path.

I've now verified all concrete claims in `plan.md` and `runbook.md` against the code. Summary of my cross-checks:

****Verified ACCURATE:**** migrations ``user_version=5``; log event names (``video_dedup_document_id``, ``video_dedup_fingerprint``, ``config_updated``, etc.); all commands (``/channels``, ``/watch``, ``/unwatch``, ``/watchlist``, ``/backfill``, ``/stop``, ``/why``, ``/show config|commands``, ``/set ...exclude...``); `FilterConfig` defaults (``exclude_gif=True``, ``exclude_round=True``, ``exclude_nosound=False``); pipeline runtime order (`topic?url-reputation?caption-safety?deep-inspection`); `deep-inspect` drops items when unwired; ``/set`` hot-reloads engine (no restart); ``/why`` account-scoping & escaping; ADR 0012 F1?F7 issue/PR numbers + #56 deferral; env vars

```
(`TG_API_ID`/`TG_API_HASH`/`DB_PATH`/`SESSIONS_DIR`); content_items SQL columns; policies  
`is_default`/`account_id`; delivery modes/targets; `filter_config`?'policies' shim.
```

****The one defect:**** runbook S6's recommended `set\_delivery\_target.py` helper is broken against the current schema.

Let me record the finding.

60. user (2026-07-07T03:41:25.840Z)

Structured output provided successfully